

ANGULAR

COMPLETE GUIDE

A Comprehensive Frontend Framework Reference

Interview Q&As • Core Concepts • Real-World Patterns

2025 Edition

CHAPTER 1

Components & Templates

The building blocks of every Angular application — how components work and how templates bring them to life

1. What Is a Component?

A component is the fundamental building block of every Angular application. It controls a portion of the UI — called a view — and is made up of three tightly coupled pieces: a TypeScript class that holds the logic and data, an HTML template that defines the structure of the view, and optional CSS styles that scope the appearance to that component only.

Every Angular app has at least one component — the root AppComponent — and the UI is composed by nesting components in a tree structure. This mirrors the DOM tree and makes it easy to reason about which component owns which part of the interface.

Core rule: A component = Class (logic) + Template (view) + Styles (appearance). These three pieces are always developed together but kept in separate files for maintainability.

2. Creating a Component

The cleanest way to create a component is via the Angular CLI. It generates the class, template, stylesheet, and spec file, and (in older setups) registers it in the nearest NgModule automatically.

```
# Generate a component using Angular CLI
ng generate component user-profile
# shorthand
ng g c user-profile

# Output:
# src/app/user-profile/user-profile.component.ts
# src/app/user-profile/user-profile.component.html
# src/app/user-profile/user-profile.component.scss
# src/app/user-profile/user-profile.component.spec.ts
```

The generated TypeScript class uses the `@Component` decorator to attach Angular metadata to the plain class:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-profile',    // HTML tag used in templates
  standalone: true,               // Angular 17+ default
  templateUrl: './user-profile.component.html',
  styleUrls: ['./user-profile.component.scss'],
})
export class UserProfileComponent {
  name = 'Gowtham';
  age = 28;
}
```

3. The @Component Decorator

The @Component decorator is how Angular knows that a plain TypeScript class should be treated as a component. It accepts a metadata object with several key properties:

- **selector** — the custom HTML tag that represents this component in other templates (e.g., <app-user-profile>)
- **templateUrl** — relative path to the external HTML template file
- **template** — inline HTML template string (alternative to templateUrl for very small components)
- **styleUrl** / **styleUrls** — path(s) to the component's stylesheet(s)
- **standalone** — when true (default in Angular 17+), the component does not need an NgModule
- **imports** — array of other standalone components, directives, or pipes this component uses
- **changeDetection** — ChangeDetectionStrategy.OnPush for performance optimization

Angular 17+ made standalone: true the default. Standalone components import their own dependencies directly instead of relying on a shared NgModule, making them more self-contained and tree-shakable.

4. Templates & Data Binding

An Angular template is standard HTML enhanced with Angular-specific syntax. It is the view layer of a component. Angular provides four types of data binding to connect the template with the component class:

4.1 Interpolation — {{ }}

Embeds a component property's value into the HTML as text. Angular evaluates the expression between the double curly braces and converts the result to a string.

```
// component class
title = 'Angular Guide';
price = 49.99;

<!-- template -->
<h1>{{ title }}</h1>
<p>Price: {{ price | currency }}</p>
<p>Today: {{ today | date:'mediumDate' }}</p>
```

4.2 Property Binding — []

Flows data from the component class INTO a DOM element property or child component Input. The expression on the right is evaluated and assigned to the target property.

```
isDisabled = true;
imageUrl    = 'https://example.com/photo.png';

<!-- Square brackets bind to a DOM property -->
<button [disabled]="isDisabled">Submit</button>
<img [src]="imageUrl" [alt]="title">

<!-- Binding to a child component Input -->
<app-card [title]="title" [count]="itemCount"></app-card>
```

4.3 Event Binding — ()

Listens for DOM events and calls a component method when that event fires. The `$event` variable carries the native DOM event object.

```
// component
onSubmit(event: Event) { console.log('Submitted', event); }
onKeyUp(value: string) { console.log('Typed:', value); }

<!-- Round brackets bind to a DOM event -->
<button (click)="onSubmit($event)">Submit</button>
<input (keyup)="onKeyUp($event.target.value)">
<form (ngSubmit)="onSubmit($event)">...</form>
```

4.4 Two-Way Binding — [()]

Combines property and event binding: data flows both from class to template AND from template back to class. This is called the 'banana in a box' syntax and is most commonly used with `ngModel` for form inputs.

```
import { FormsModule } from '@angular/forms';

// component
username = '';

<!-- Two-way: class -> view AND view -> class -->
```

```
<input [(ngModel)]="username" placeholder="Enter name">
<p>Hello, {{ username }}!</p>
```

5. Input & Output — Component Communication

Components communicate with each other using `@Input()` and `@Output()` decorators. This is the primary pattern for passing data between parent and child components.

5.1 @Input() — Parent to Child

```
// child component: product-card.component.ts
import { Component, Input } from '@angular/core';

@Component({ selector: 'app-product-card', standalone: true, ... })
export class ProductCardComponent {
  @Input() productName!: string;
  @Input() price!: number;
  @Input({ required: true }) id!: number; // Angular 16+: mark as required
}

<!-- parent template -->
<app-product-card
  [productName]='''Angular Book'''
  [price]="29.99"
  [id]="101">
</app-product-card>
```

5.2 @Output() — Child to Parent

```
// child component
import { Component, Output, EventEmitter } from '@angular/core';

@Component({ selector: 'app-product-card', standalone: true, ... })
export class ProductCardComponent {
  @Output() addToCart = new EventEmitter<number>();

  onAdd(productId: number) {
    this.addToCart.emit(productId);
  }
}

<!-- parent template -->
<app-product-card (addToCart)="handleAddToCart($event)"></app-product-card>
```

6. Built-in Directives

Angular ships with several structural and attribute directives that let you control the DOM declaratively from inside the template. Angular 17 introduced a new built-in control flow syntax (@if, @for, @switch) as an alternative to the older *ngIf / *ngFor directives.

6.1 @if / *ngIf — Conditional Rendering

```
<!-- New syntax (Angular 17+) -->
@if (isLoggedIn) {
  <p>Welcome back, {{ user.name }}!</p>
} @else {
  <p>Please log in.</p>
}

<!-- Classic syntax -->
<p *ngIf="isLoggedIn; else loginBlock">Welcome back!</p>
<ng-template #loginBlock><p>Please log in.</p></ng-template>
```

6.2 @for / *ngFor — List Rendering

```
<!-- New syntax (Angular 17+) - track is REQUIRED -->
@for (item of items; track item.id) {
  <li>{{ item.name }} - {{ item.price | currency }}</li>
} @empty {
  <li>No items found.</li>
}

<!-- Classic syntax -->
<li *ngFor="let item of items; trackBy: trackById">
  {{ item.name }}
</li>
```

6.3 [ngClass] and [ngStyle]

```
// component
isActive = true;
hasError = false;

<!-- ngClass: conditionally add/remove CSS classes -->
<div [ngClass]="{ 'active': isActive, 'error': hasError }">
  Content
</div>

<!-- ngStyle: bind inline styles dynamically -->
<p [ngStyle]="{ 'color': isActive ? 'green' : 'red', 'font-size.px': 16 }">
  Status
</p>
```

7. Component Lifecycle Hooks

Angular components go through a well-defined lifecycle from creation to destruction. Lifecycle hooks are methods you can implement on the component class to run code at specific stages.

- `ngOnChanges()` — called before `ngOnInit` and whenever an `@Input` property changes; receives a `SimpleChanges` object
- `ngOnInit()` — called once after the first `ngOnChanges`; ideal for component initialization and data fetching
- `ngDoCheck()` — called during every change detection run; use for custom change detection logic
- `ngAfterViewInit()` — called once after the component's view and child views are fully initialized
- `ngAfterViewChecked()` — called after every check of the component's view and child views
- `ngOnDestroy()` — called just before Angular destroys the component; ideal for unsubscribing observables

```
import { Component, OnInit, OnDestroy, Input, OnChanges, SimpleChanges } from
 '@angular/core';
import { Subscription } from 'rxjs';

@Component({ selector: 'app-timer', standalone: true, template: '...' })
export class TimerComponent implements OnInit, OnChanges, OnDestroy {
  @Input() label = 'Timer';
  private sub!: Subscription;

  ngOnChanges(changes: SimpleChanges) {
    console.log('Input changed:', changes['label']);
  }

  ngOnInit() {
    // safe to access @Inputs here
    this.sub = someObservable$.subscribe();
  }

  ngOnDestroy() {
    this.sub.unsubscribe(); // prevent memory leaks
  }
}
```

8. ViewChild & ContentChild

Angular provides `@ViewChild` and `@ContentChild` decorators to get a direct reference to a child element, component, or directive from within the parent class.

```
import { Component, ViewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({
```

```
standalone: true,
template: `
  <input #nameInput type="text">
  <app-child #childComp></app-child>
`
})
export class ParentComponent implements AfterViewInit {
  @ViewChild('nameInput') inputEl!: ElementRef<HTMLInputElement>;
  @ViewChild('childComp') child!: ChildComponent;

  ngAfterViewInit() {
    this.inputEl.nativeElement.focus(); // focus input on init
    this.child.doSomething();           // call child method
  }
}
```

Interview Questions & Answers

Q1: What is an Angular component and what are its three main parts?

- A component is the core building block of Angular that controls a section of the UI (a view)
- The three parts are: (1) a TypeScript class that holds data and logic, (2) an HTML template that defines the structure, and (3) CSS styles scoped to that component
- They are linked together via the `@Component` decorator, which supplies Angular with the selector, templateUrl, and styleUrls metadata
- Angular renders a tree of components — the root AppComponent sits at the top, and all other components are nested beneath it

Q2: What is the difference between property binding and interpolation?

- Interpolation `{{ }}` converts a component expression to a string and inserts it as text content inside an element
- Property binding `[ ]` sets a DOM element property or a child component `@Input` to the value of a component expression — it does not have to be a string
- Use interpolation for displaying values as text; use property binding when the target is a non-string property (e.g., `[disabled]`, `[src]`, `[href]`)
- Example: `[disabled]="isLoading"` is correct; `disabled="{{ isLoading }}"` sets a string attribute, not a boolean property

Q3: What are `@Input()` and `@Output()` used for?

- `@Input()` allows a parent component to pass data DOWN to a child component via property binding
- `@Output()` allows a child component to send events UP to a parent via an EventEmitter
- This one-directional parent-to-child, child-to-parent communication is the standard Angular pattern for component interaction

- From Angular 16+, you can mark an `@Input` as required: `@Input({ required: true })` which throws a compile-time error if not supplied

Q4: What is the difference between `*ngIf` and `@if`?

- `*ngIf` is the classic structural directive syntax: it creates/destroys DOM elements based on a condition and requires `CommonModule` or `NgIf` to be imported
- `@if` is the new built-in control flow syntax introduced in Angular 17 — it is compiled into the template at build time, no import required
- `@if` supports `@else` and `@else if` blocks inline without needing an `ng-template`, making the template more readable
- The `@if` syntax also performs slightly better because Angular can optimize it at compile time rather than at runtime

Q5: Why is `trackBy` (or `track`) important in `*ngFor` / `@for`?

- Without `trackBy`/`track`, Angular re-renders the entire list DOM whenever the array reference changes, even if only one item changed
- `trackBy` provides Angular with a unique identity function per item, so it can reuse existing DOM nodes and only update what actually changed
- In `@for`, `track` is required: `@for (item of items; track item.id)` — the compiler enforces it
- This significantly improves performance in large lists and avoids input focus loss, animation resets, and scroll position jumps

Q6: What is the component lifecycle and which hooks are most commonly used?

- Angular calls lifecycle hooks in a defined order: `ngOnChanges` -> `ngOnInit` -> `ngDoCheck` -> `ngAfterViewInit` -> `ngAfterViewChecked` -> `ngOnDestroy`
- `ngOnInit` is the most commonly used: it runs once after all `@Inputs` are set and is the right place for initialization logic and HTTP calls
- `ngOnChanges` is used when you need to react to `@Input` property changes and receives a `SimpleChanges` map showing old and new values
- `ngOnDestroy` is critical for cleanup: always unsubscribe from Observables, clear timers, and detach event listeners here to prevent memory leaks

Q7: What is the difference between a standalone component and an `NgModule`-based component?

- An `NgModule`-based component must be declared in an `NgModule`'s declarations array to be usable; it relies on the module to import shared dependencies like `CommonModule`
- A standalone component (`standalone: true`) declares its own imports directly in the `@Component` metadata — it is self-sufficient
- Standalone is the default in Angular 17+; it reduces boilerplate, improves tree-shaking, and makes components easier to lazy-load

- Standalone components can be mixed with NgModule-based components — they can be imported into NgModules and vice versa

Q8: What is ViewChild and when would you use it?

- @ViewChild gives you a direct TypeScript reference to a child component, directive, or DOM element defined in the same component's template
- It is available after ngAfterViewInit() — never access it in ngOnInit because the view has not been initialized yet
- Common use cases: programmatically focusing an input, calling a method on a child component, or accessing a canvas element for manual rendering
- @ContentChild is similar but queries projected content (ng-content), while @ViewChild queries the component's own template

Q9: What are the four types of data binding in Angular?

- Interpolation {{ expression }} — one-way, class to template, renders values as text
- Property Binding [property]="expression" — one-way, class to template, sets DOM/component properties
- Event Binding (event)="handler(\$event)" — one-way, template to class, responds to DOM events
- Two-Way Binding [(ngModel)]="property" — both directions at once; shorthand for combining [ngModel] and (ngModelChange). Requires FormsModule

Q10: What is the selector in @Component and what are its types?

- The selector is the CSS selector Angular uses to identify where to render the component in a template
- Element selector (most common): selector: 'app-user-card' — used as <app-user-card> in templates
- Attribute selector: selector: '[appHighlight]' — used as <div appHighlight> — common for directives
- Class selector: selector: '.card-widget' — used as <div class='card-widget'> — rarely used for components

Directives

Extend HTML with custom behaviour — structural, attribute, and component directives explained

1. What Is a Directive?

A directive is a class that adds behaviour to elements in the DOM. In Angular, directives are one of the three types of Angular artifacts that can appear in templates — the other two being components and pipes. In fact, a component is technically a directive with a template.

Angular has three categories of directive: Component Directives (which have a template), Structural Directives (which change the DOM layout by adding or removing elements), and Attribute Directives (which change the appearance or behaviour of an existing element without adding or removing it).

Key insight: Components are directives with templates. When you write `@Component`, you are creating a directive that also controls a view. All three directive types — component, structural, and attribute — share the same underlying Directive system.

2. Structural Directives

Structural directives alter the DOM structure by adding, removing, or replacing elements. They are prefixed with an asterisk (*) in templates, which is Angular shorthand for wrapping the element in an `<ng-template>`. Angular 17+ introduced a new built-in control flow syntax (`@if`, `@for`, `@switch`) as the recommended alternative.

2.1 `*ngIf` / `@if` — Conditional Rendering

Conditionally includes or excludes a block of the DOM based on a boolean expression. When the condition is false, the element and all its children are completely removed from the DOM — not just hidden.

```
// component.ts
isAdmin    = true;
isLoading  = false;
user       = { name: 'Gowtham', role: 'admin' };

<!-- ✅ New @if syntax (Angular 17+) - no import needed -->
@if (isLoading) {
  <app-spinner></app-spinner>
} @else if (isAdmin) {
```

```
<p>Welcome, Admin {{ user.name }}!</p>
} @else {
  <p>Access restricted.</p>
}

<!-- Classic *ngIf with else template reference -->
<div *ngIf="isAdmin; else guestBlock">
  <p>Admin Panel</p>
</div>
<ng-template #guestBlock>
  <p>Guest View</p>
</ng-template>
```

2.2 *ngFor / @for — List Rendering

Iterates over a collection and creates a DOM element for each item. The track expression (required in @for) tells Angular how to identify each item uniquely, enabling efficient DOM reuse when the list changes.

```
// component.ts
products = [
  { id: 1, name: 'Laptop', price: 999 },
  { id: 2, name: 'Monitor', price: 399 },
  { id: 3, name: 'Keyboard', price: 79 },
];

<!--  New @for syntax — track is required -->
<ul>
  @for (product of products; track product.id) {
    <li>
      <strong>{{ product.name }}</strong> — {{ product.price | currency }}
    </li>
  } @empty {
    <li>No products available.</li>
  }
</ul>

<!-- Classic *ngFor with index and trackBy -->
<li *ngFor="let p of products; let i = index; trackBy: trackById">
  {{ i + 1 }}. {{ p.name }}
</li>

// component.ts — trackBy function for classic syntax
trackById(index: number, item: { id: number }) {
  return item.id;
}
```

2.3 *ngSwitch / @switch — Multiple Conditions

Renders one of several possible views based on a matching expression — similar to a JavaScript switch statement. Cleaner than chaining multiple @if/@else when comparing a single value against many options.

```
// component.ts
userRole = 'editor'; // 'admin' | 'editor' | 'viewer'

<!--  New @switch syntax -->
@switch (userRole) {
  @case ('admin') {
    <app-admin-dashboard></app-admin-dashboard>
  }
  @case ('editor') {
    <app-editor-panel></app-editor-panel>
  }
  @default {
    <app-read-only-view></app-read-only-view>
  }
}

<!-- Classic ngSwitch -->
<div [ngSwitch]="userRole">
  <app-admin-dashboard *ngSwitchCase="'admin'"></app-admin-dashboard>
  <app-editor-panel *ngSwitchCase="'editor'"></app-editor-panel>
  <app-read-only-view *ngSwitchDefault></app-read-only-view>
</div>
```

3. Attribute Directives

Attribute directives change the appearance or behaviour of an existing DOM element, component, or another directive. Unlike structural directives, they do not add or remove elements — they only modify what is already there.

3.1 NgClass — Dynamic CSS Classes

```
// component.ts
isActive = true;
hasError = false;
isLoading = false;

<!-- Object syntax: key = class name, value = boolean condition -->
<div [ngClass]="{
  'btn-active': isActive,
  'btn-error': hasError,
  'btn-loading': isLoading
}">Submit</div>

<!-- Array syntax: conditionally include class strings -->
<div [ngClass]="[isActive ? 'active' : 'inactive', 'base-class']">
  Content
</div>

<!-- String syntax: always apply a fixed class -->
<div [ngClass]="'card shadow rounded'">Card</div>
```

3.2 NgStyle — Dynamic Inline Styles

```
// component.ts
progress    = 75;    // 0-100
textColor   = '#333';
fontSize    = 16;

<!-- Object with CSS property names -->
<div [ngStyle]="{
  'width.%':      progress,
  'background-color': progress > 50 ? 'green' : 'red',
  'color':        textColor,
  'font-size.px': fontSize
}">
  Progress: {{ progress }}%
</div>

<!-- Using a computed getter -->
<p [ngStyle]="statusStyles">Status</p>

// component.ts
get statusStyles() {
  return { color: this.hasError ? 'red' : 'green', 'font-weight': 'bold' };
}
```

4. Building a Custom Attribute Directive

You can create your own attribute directives to encapsulate reusable DOM behaviour. A directive class uses the `@Directive` decorator and injects `ElementRef` to access the host DOM element. The `HostListener` decorator lets you listen to DOM events on the host.

4.1 Basic Highlight Directive

```
// highlight.directive.ts
import { Directive, ElementRef, HostListener, Input, OnInit } from
  '@angular/core';

@Directive({
  selector: '[appHighlight]', // attribute selector
  standalone: true
})
export class HighlightDirective implements OnInit {
  @Input() appHighlight = 'yellow'; // input doubles as directive name
  @Input() defaultColor = 'white';

  constructor(private el: ElementRef) {}

  ngOnInit() {
    this.setBackground(this.defaultColor);
  }
}
```

```
@HostListener('mouseenter')
onMouseEnter() { this.setBackground(this.appHighlight); }

@HostListener('mouseleave')
onMouseLeave() { this.setBackground(this.defaultColor); }

private setBackground(color: string) {
  this.el.nativeElement.style.backgroundColor = color;
}
}

<!-- Usage in template -->
<p appHighlight="lightblue" defaultColor="#f9f9f9">
  Hover to highlight me!
</p>
```

4.2 Click-Outside Directive (Real-World Example)

A common real-world use case is detecting clicks outside a dropdown or modal to close it. This requires listening to document-level events from inside the directive using `@HostListener` with the document target.

```
// click-outside.directive.ts
import { Directive, ElementRef, EventEmitter, HostListener, Output } from
 '@angular/core';

@Directive({
  selector: '[appClickOutside]',
  standalone: true
})
export class ClickOutsideDirective {
  @Output() clickOutside = new EventEmitter<void>();

  constructor(private el: ElementRef) {}

  @HostListener('document:click', ['$event.target'])
  onClick(target: HTMLElement) {
    const isInside = this.el.nativeElement.contains(target);
    if (!isInside) {
      this.clickOutside.emit();
    }
  }
}

<!-- Usage: close dropdown when clicking outside -->
<div appClickOutside (clickOutside)="isOpen = false">
  <button (click)="isOpen = !isOpen">Toggle Dropdown</button>
  @if (isOpen) {
    <ul class="dropdown">
      <li>Option 1</li>
      <li>Option 2</li>
    </ul>
  }
</div>
```

5. Building a Custom Structural Directive

Custom structural directives use `TemplateRef` and `ViewContainerRef` to conditionally add or remove DOM blocks. The asterisk (*) syntax in templates is syntactic sugar that Angular desugars into an `<ng-template>` with the directive applied.

```
// unless.directive.ts - opposite of *ngIf
import { Directive, Input, TemplateRef, ViewContainerRef } from '@angular/core';

@Directive({
  selector: '[appUnless]',
  standalone: true
})
export class UnlessDirective {
  private hasView = false;

  constructor(
    private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef
  ) {}

  @Input() set appUnless(condition: boolean) {
    if (!condition && !this.hasView) {
      // condition is false - show the template
      this.viewContainer.createEmbeddedView(this.templateRef);
      this.hasView = true;
    } else if (condition && this.hasView) {
      // condition is true - hide the template
      this.viewContainer.clear();
      this.hasView = false;
    }
  }
}

<!-- Usage: shows the block UNLESS isLoggedIn is true -->
<p *appUnless="isLoggedIn">Please log in to continue.</p>
```

6. HostBinding — Binding to Host Element Properties

`@HostBinding` lets a directive bind directly to a property or attribute of the element it is applied to, without needing to use `ElementRef` manually. This is the cleaner, Angular-idiomatic approach for modifying the host element.

```
// tooltip.directive.ts
import { Directive, HostBinding, HostListener, Input } from '@angular/core';

@Directive({
  selector: '[appTooltip]',
  standalone: true
```

```
})
export class TooltipDirective {
  @Input() appTooltip = '';

  // Binds to host element's 'title' attribute
  @HostBinding('attr.title') get tooltipText() {
    return this.appTooltip;
  }

  // Binds to host element's CSS class
  @HostBinding('class.has-tooltip') hasTooltipClass = true;

  // Binds to a style property
  @HostBinding('style.cursor') cursor = 'help';
}

<!-- Usage -->
<span appTooltip="This is a tooltip message">Hover me</span>
<!-- Renders as: <span title='...' class='has-tooltip' style='cursor:help'> -->
```

7. Directive Composition API (Angular 15+)

The Directive Composition API (Angular 15+) lets you apply multiple directives to a component or another directive via the `hostDirectives` property in `@Component` or `@Directive`. This promotes reuse without requiring inheritance.

```
// reusable directives
import { HighlightDirective } from './highlight.directive';
import { ClickOutsideDirective } from './click-outside.directive';

@Component({
  selector: 'app-dropdown',
  standalone: true,
  template: `<ng-content></ng-content>`,
  // Compose multiple directives onto this component
  hostDirectives: [
    {
      directive: HighlightDirective,
      inputs: ['appHighlight'], // expose the input
    },
    {
      directive: ClickOutsideDirective,
      outputs: ['clickOutside'], // expose the output
    }
  ]
})
export class DropdownComponent {}

<!-- The composed component automatically has both directive behaviours -->
<app-dropdown appHighlight="cyan" (clickOutside)="close()">
  Dropdown content
</app-dropdown>
```

Interview Questions & Answers

Q1: What are the three types of directives in Angular?

- Component Directives — directives with a template; they define a view and a selector used as an HTML tag (most commonly used type)
- Structural Directives — change the DOM layout by adding or removing elements; marked with `*` in templates (e.g., `*ngIf`, `*ngFor`)
- Attribute Directives — change the appearance or behaviour of an element without changing the DOM structure (e.g., `NgClass`, `NgStyle`, custom highlights)
- A component is technically a directive — `@Component` extends `@Directive` and adds the template and styles metadata

Q2: What does the asterisk (*) in `*ngIf` and `*ngFor` actually do?

- The `*` is syntactic sugar Angular uses to transform the shorthand into a full `<ng-template>` element
- `*ngIf="condition"` becomes: `<ng-template [ngIf]="condition"><element></element></ng-template>`
- `*ngFor="let item of items"` becomes: `<ng-template ngFor let-item [ngForOf]="items"><element></element></ng-template>`
- Angular desugars all `*` directives at compile time before the template is rendered, so there is no runtime cost for this transformation

Q3: What is the difference between `*ngIf` and `[hidden]`?

- `*ngIf` completely adds or removes the element from the DOM — when false, the element and all its children do not exist in the DOM at all
- `[hidden]="true"` keeps the element in the DOM but applies CSS `display:none` — it still occupies memory and runs change detection
- Use `*ngIf` when the element is conditionally needed: it is better for performance and prevents child components from initialising unnecessarily
- Use `[hidden]` only when you need to preserve the element's state (e.g., an input's scroll position or a video player's progress) across toggles

Q4: Why is `trackBy` important in `*ngFor` and why is `track` required in `@for`?

- By default, `*ngFor` identifies list items by their object reference; when the array is reassigned (e.g., from an API), Angular destroys and recreates every DOM node
- `trackBy` returns a unique identity (usually `id`) so Angular can match old DOM nodes to new data and only update the nodes that actually changed
- The `@for` syntax made `track` mandatory because it is such a critical performance best practice — the compiler enforces it at build time
- Result: fewer DOM operations, preserved input focus, no animation resets, and significantly better rendering performance on large lists

Q5: How do you create a custom attribute directive?

- Create a class decorated with `@Directive({ selector: '[appMyDirective]', standalone: true })`
- Inject `ElementRef` in the constructor to access the raw DOM element of the host
- Use `@HostListener('event')` methods to listen to DOM events on the host element
- Use `@Input()` to accept configuration from the parent template, and `@HostBinding()` to bind properties back to the host element without touching `ElementRef` directly

Q6: What is the difference between `@HostListener` and `@HostBinding`?

- `@HostListener` subscribes to DOM events on the host element — it is used to react to events like click, mouseenter, keydown on the element the directive is applied to
- `@HostBinding` binds a directive property to a host element property, attribute, or CSS class — it flows data from the directive OUT to the host element
- Example: `@HostBinding('class.active')` `isActive = true` will add the active CSS class to the host element whenever `isActive` is true
- Together they let a directive fully control its host element's appearance and behaviour without requiring direct DOM manipulation via `ElementRef`

Q7: What are `TemplateRef` and `ViewContainerRef` and why are they needed for structural directives?

- `TemplateRef` represents the embedded template (the content wrapped in `<ng-template>`) — it holds the blueprint of what to render
- `ViewContainerRef` is a container in the DOM where Angular can dynamically create, insert, or remove views (rendered template instances)
- A structural directive injects both: it uses `TemplateRef` as the blueprint and `ViewContainerRef.createEmbeddedView()` to stamp instances into the DOM
- `ViewContainerRef.clear()` removes all views from the container — this is how `*ngIf` removes the element when the condition becomes false

Q8: What is the `NgClass` directive and when would you use it over class binding?

- `NgClass` dynamically adds or removes CSS classes on an element based on an object map, array, or string expression
- Single class binding `[class.active]="isActive"` is simpler and preferred when toggling a single class — it is more readable
- `[ngClass]` is preferred when toggling multiple classes at once based on different conditions — a single `[ngClass]` object replaces multiple `[class.x]` bindings
- Both are equivalent under the hood — `[ngClass]` is just more ergonomic when managing several conditional classes simultaneously

Q9: What is the Directive Composition API and what problem does it solve?

- Introduced in Angular 15, the Directive Composition API lets you apply multiple directives to a component or directive via the `hostDirectives` property in `@Component/@Directive` metadata
- It solves the problem of shared behaviour: before this, you had to choose between copy-pasting directive logic or using inheritance (which is fragile and inflexible)
- With `hostDirectives`, a component inherits the behaviour of one or more directives without the user of the component needing to apply them manually in the template
- You can selectively expose or hide the directive's `@Input` and `@Output` via the `inputs` and `outputs` arrays in the `hostDirectives` config

Q10: What is the difference between an attribute selector and an element selector for a directive?

- Element selector: selector: `'app-card'` — the directive (or component) is applied by using it as an HTML tag: `<app-card></app-card>`
- Attribute selector: selector: `'[appHighlight]'` — the directive is applied as an attribute on an existing element: `<p appHighlight></p>`
- Attribute selectors are the standard for pure directives (non-components) because they decorate existing elements without replacing them
- You can also combine selectors: selector: `'button[appRipple]'` will only match buttons that also have the `appRipple` attribute, preventing accidental application

Services & Dependency Injection

Centralise logic, share data, and let Angular wire dependencies together automatically

1. What Is a Service?

A service is a plain TypeScript class that holds logic or data that does not belong in a component. Components should focus on the user interface — displaying data and responding to events. Everything else — fetching data, business logic, caching, logging, authentication state — belongs in a service.

Services are designed to be reusable and injectable. Once created, Angular's Dependency Injection system can provide a single shared instance of the service to any component, directive, or other service that requests it.

Rule of thumb: if two or more components need the same data or logic, that logic belongs in a service. If you find yourself duplicating code across components, extract it into a service.

2. Creating a Service

The Angular CLI generates a service class with the `@Injectable` decorator already applied. This decorator tells Angular that the class can participate in the DI system — both as a dependency that can be injected into others, and as a consumer that can receive its own injected dependencies.

```
# Generate a service via Angular CLI
ng generate service cart
# shorthand
ng g s cart

# Output:
# src/app/cart.service.ts
# src/app/cart.service.spec.ts
```

```
// cart.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // ← registers this service with the root injector
})
```

```
export class CartService {
  private items: string[] = [];

  addItem(item: string): void {
    this.items.push(item);
  }

  getItems(): string[] {
    return [...this.items];
  }

  clearCart(): void {
    this.items = [];
  }
}
```

3. What Is Dependency Injection?

Dependency Injection (DI) is a design pattern where a class declares what it needs (its dependencies) rather than creating those dependencies itself. Angular's DI framework reads those declarations and automatically provides the correct instances at runtime.

The key benefit is decoupling: a component does not know or care where its service comes from or how it is constructed. Angular handles instantiation, caching, and lifetime management. This makes components easier to test, because you can substitute real services with mock versions during testing.

```
// WITHOUT DI - tightly coupled, hard to test
export class ProductListComponent {
  private productService = new ProductService(); // ❌ creates its own instance
}

// WITH DI - loosely coupled, easily testable
export class ProductListComponent {
  constructor(private productService: ProductService) {} // ✅ Angular injects it
}

// Modern inject() function (Angular 14+) - works outside constructors too
import { inject } from '@angular/core';

export class ProductListComponent {
  private productService = inject(ProductService); // ✅ field injection
}
```

4. Injectors & the providedIn Property

Angular maintains a hierarchy of injectors. When a component requests a service, Angular walks up the injector tree until it finds a provider for that service. Understanding this hierarchy is key to controlling whether a service is shared globally or scoped to a specific feature.

4.1 providedIn: 'root' — Singleton across the whole app

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  // One single instance shared by every component in the application.
  // Tree-shakable: if no component injects AuthService, it is removed
  // from the production bundle automatically.
  private currentUser: User | null = null;

  login(user: User) { this.currentUser = user; }
  logout()           { this.currentUser = null; }
  getUser()          { return this.currentUser; }
}
```

4.2 Component-level providers — Scoped instance

```
// Providing a service at the component level creates a FRESH instance
// for that component and all its children. Other components get a
// different instance — state is NOT shared.
@Component({
  selector: 'app-checkout',
  standalone: true,
  templateUrl: './checkout.component.html',
  providers: [CheckoutService] // ← scoped to this component subtree
})
export class CheckoutComponent {
  private checkout = inject(CheckoutService);
}
```

4.3 providedIn: 'any' and Route-level providers

```
// Route-level provider — new instance per lazy-loaded route module
export const CHECKOUT_ROUTES: Routes = [
  {
    path: 'checkout',
    loadComponent: () => import('./checkout.component'),
    providers: [CheckoutService] // fresh instance for this route
  }
];

// providedIn: 'any' — new instance per lazy-loaded injector context
@Injectable({ providedIn: 'any' })
export class FeatureService {}
```

providedIn: 'root' is the recommended default for most services. It creates a single shared instance, is tree-shakable, and avoids the boilerplate of registering services in providers arrays.

5. The inject() Function — Modern DI

Angular 14 introduced the `inject()` function as a modern alternative to constructor injection. It can be used in component fields, factory functions, and custom injection contexts — making it particularly useful in standalone components and functional guards/interceptors.

```
import { Component, inject } from '@angular/core';
import { UserService }      from '../user.service';
import { CartService }      from '../cart.service';
import { Router }           from '@angular/router';

@Component({ selector: 'app-header', standalone: true, template: '...' })
export class HeaderComponent {
  // ✅ Field injection – cleaner than constructor for many services
  private userService = inject(UserService);
  private cartService = inject(CartService);
  private router       = inject(Router);

  get cartCount() { return this.cartService.getItems().length; }
  get userName()  { return this.userService.currentUser?.name; }

  logout() {
    this.userService.logout();
    this.router.navigate(['/login']);
  }
}
```

6. Services with HttpClient — Real-World Example

The most common use of services is wrapping HTTP calls. Angular's `HttpClient` is itself injectable, and you inject it into your service. This keeps all API interaction in one place — components only call service methods and receive data, never making HTTP calls themselves.

```
// product.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient }         from '@angular/common/http';
import { Observable }         from 'rxjs';
import { catchError, map, tap } from 'rxjs/operators';

export interface Product {
  id: number;
  name: string;
  price: number;
}

@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);
  private baseUrl = 'https://api.example.com/products';
```

```
getAll(): Observable<Product[]> {
  return this.http.get<Product[]>(this.baseUrl).pipe(
    tap(products => console.log('Fetched:', products.length)),
    catchError(err => { console.error(err); throw err; })
  );
}

getById(id: number): Observable<Product> {
  return this.http.get<Product>(`${this.baseUrl}/${id}`);
}

create(product: Omit<Product, 'id'>): Observable<Product> {
  return this.http.post<Product>(this.baseUrl, product);
}

update(id: number, changes: Partial<Product>): Observable<Product> {
  return this.http.patch<Product>(`${this.baseUrl}/${id}`, changes);
}

delete(id: number): Observable<void> {
  return this.http.delete<void>(`${this.baseUrl}/${id}`);
}
}
```

```
// product-list.component.ts - component consumes the service cleanly
import { Component, inject, OnInit } from '@angular/core';
import { AsyncPipe } from '@angular/common';
import { ProductService, Product } from '../product.service';
import { Observable } from 'rxjs';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [AsyncPipe],
  template: `
    @if (products$ | async; as products) {
      @for (p of products; track p.id) {
        <div class='card'>{{ p.name }} - {{ p.price | currency }}</div>
      }
    }
  `,
})
export class ProductListComponent implements OnInit {
  private productService = inject(ProductService);
  products$: Observable<Product[]>;

  ngOnInit() {
    this.products$ = this.productService.getAll();
  }
}
```

7. Injection Tokens — Injecting Non-Class Values

Sometimes you need to inject a value that is not a class — a configuration object, a string, or an interface. Angular's `InjectionToken` creates a unique DI key for these cases.

```
// tokens.ts — define the token
import { InjectionToken } from '@angular/core';

export interface AppConfig {
  apiUrl: string;
  maxRetries: number;
}

export const APP_CONFIG = new InjectionToken<AppConfig>('app.config');

// main.ts — provide the value at bootstrap
bootstrapApplication(AppComponent, {
  providers: [
    {
      provide: APP_CONFIG,
      useValue: {
        apiUrl: 'https://api.example.com',
        maxRetries: 3
      }
    }
  ]
});

// any service or component — inject the token
import { inject } from '@angular/core';
import { APP_CONFIG } from './tokens';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private config = inject(APP_CONFIG);

  getBaseUrl() { return this.config.apiUrl; }
}
```

8. Provider Types — `useClass`, `useValue`, `useFactory`, `useExisting`

Angular's providers array supports several provider definitions that give you fine-grained control over how a dependency is created and resolved.

```
// 1. useClass — provide a different class for an abstract token (e.g. for testing)
{ provide: LoggerService, useClass: ConsoleLoggerService }
{ provide: LoggerService, useClass: environment.production ? RemoteLoggerService : ConsoleLoggerService }

// 2. useValue — provide a static value (config, constants, mocks)
```

```
{ provide: APP_CONFIG, useValue: { apiUrl: 'https://api.example.com' } }

// 3. useFactory - compute the value dynamically at injection time
{
  provide: LoggerService,
  useFactory: (config: AppConfig) => {
    return config.production ? new RemoteLoggerService() : new
    ConsoleLoggerService();
  },
  deps: [APP_CONFIG] // declare dependencies of the factory
}

// 4. useExisting - alias one token to another already-registered token
{ provide: NewAuthService, useExisting: LegacyAuthService }
```

Interview Questions & Answers

Q1: What is a service in Angular and why do we need it?

- A service is a TypeScript class decorated with `@Injectable` that holds reusable logic or shared state that does not belong in a component
- Components should only handle UI concerns (displaying data, user interaction). HTTP calls, business rules, caching, and state management belong in services
- Services are singleton by default (with `providedIn: 'root'`) — one instance is shared across all components that inject it, keeping state consistent
- They make code testable: you can inject a mock service in tests without changing the component

Q2: What is Dependency Injection and how does Angular implement it?

- DI is a pattern where a class declares what it needs (dependencies) instead of creating them itself — the framework resolves and provides them
- Angular maintains a hierarchical tree of injectors. When a class requests a dependency, Angular walks up the tree to find a provider for it
- You declare dependencies either in the constructor (`constructor(private svc: MyService)`) or with the `inject()` function (`private svc = inject(MyService)`)
- The key benefits are: loose coupling, testability (easy to swap real services for mocks), and lifecycle management (Angular controls when services are created and destroyed)

Q3: What does `providedIn: 'root'` mean and why is it the recommended default?

- `providedIn: 'root'` registers the service with the application's root injector, creating a single shared instance for the entire app — a true singleton
- It is tree-shakable: if nothing in the app injects the service, Angular's build tools will remove it from the production bundle automatically

- It is simpler than the older approach of manually adding the service to an NgModule's providers array, which was error-prone and not tree-shakable
- Alternative scopes: providers on a @Component create a fresh instance per component subtree; route-level providers create a new instance per lazy-loaded route

Q4: What is the difference between constructor injection and the inject() function?

- Constructor injection: dependencies are declared as constructor parameters — Angular injects them when creating the class instance
- inject() function (Angular 14+): called at class field initialization time — cleaner syntax, works in functional contexts like guards, interceptors, and factory functions
- inject() cannot be used inside methods or lifecycle hooks — it must be called during instance construction (field initializers or constructor)
- Both are functionally equivalent for components and services; inject() is preferred in modern Angular code for its conciseness and flexibility in non-class contexts

Q5: How do you scope a service to a specific component subtree?

- Add the service class to the providers array in the @Component decorator: providers: [MyService]
- Angular creates a new, independent instance of MyService for that component and all its children — state is not shared with other parts of the app
- When the component is destroyed, Angular also destroys that scoped service instance, freeing its memory
- Use cases: a wizard flow where each step needs a fresh copy of a form state service, or a feature module where the service should not bleed into other features

Q6: What is an InjectionToken and when would you use it?

- InjectionToken creates a unique, type-safe DI key for values that are not class instances — such as configuration objects, strings, numbers, or interface types
- Classes are their own DI token because they have a runtime identity. Interfaces and primitive types do not — InjectionToken fills that gap
- Example: const API_URL = new InjectionToken<string>('api.url') can be provided with { provide: API_URL, useValue: 'https://...' } and injected with inject(API_URL)
- InjectionTokens are also used to build plugin-style architectures where multiple implementations can be collected under one token using multi: true

Q7: What is the difference between useClass, useValue, useFactory, and useExisting?

- useClass — tells Angular to instantiate a specific class when the token is requested; useful for swapping implementations (e.g., prod vs mock logger)
- useValue — provides a static, pre-created value (object, string, boolean); ideal for configuration constants and environment variables

- `useFactory` — calls a factory function to create the value at injection time; supports dynamic logic and can declare its own deps with the `deps` array
- `useExisting` — creates an alias, pointing one token to another already-registered token; both tokens resolve to the same instance

Q8: How does Angular's injector hierarchy work?

- Angular has a tree of injectors that mirrors the component tree: `EnvironmentInjector` (root) at the top, `NodeInjector` (component-level) at each component
- When a component requests a dependency, Angular first checks the component's own injector, then walks UP the tree to parent components, then to the root injector
- If no provider is found anywhere in the tree, Angular throws a `NullInjectorError`
- This hierarchy means child components can override a service for themselves and their descendants by adding it to their own `providers` array

Q9: Why should business logic and HTTP calls be in services rather than components?

- Separation of concerns: components are responsible for the view; services are responsible for data and logic — each class has a single, clear purpose
- Reusability: a service can be injected into any number of components without duplicating the logic
- Testability: a service can be unit-tested in isolation; a component can be tested by providing a mock service without hitting a real API
- Maintainability: changing an API endpoint or business rule requires updating one service, not every component that fetches that data

Q10: How do you provide `HttpClient` in a standalone Angular application?

- In a standalone Angular app (Angular 17+), you provide `HttpClient` in `main.ts` using the `provideHttpClient()` function instead of importing `HttpClientModule`
- `bootstrapApplication(AppComponent, { providers: [provideHttpClient()] })` registers the HTTP infrastructure globally
- You can add interceptors via `withInterceptors([myInterceptorFn])` as a parameter to `provideHttpClient()`
- In `NgModule`-based apps, you import `HttpClientModule` in the root `AppModule`'s `imports` array — `provideHttpClient()` is the standalone equivalent

Routing & Navigation

Navigate between views, protect routes, lazy-load features, and pass data through the URL

1. What Is the Angular Router?

The Angular Router is a powerful, built-in navigation library that maps URL paths to components and manages the browser history stack. It enables Single-Page Application (SPA) navigation — the page never fully reloads; Angular simply swaps out the active component based on the current URL.

The Router reads a route configuration array you define, matches the current URL against it, and renders the matching component inside a `<router-outlet>` placeholder in your template. It supports nested routes, lazy loading, route guards, resolvers, and query parameters out of the box.

Core concept: The Router never reloads the page. It intercepts navigation events, updates the URL in the browser address bar, and dynamically renders the correct component — all within the same page load.

2. Setting Up Routes

Routes are defined as an array of Route objects. Each route maps a URL path to a component. In a standalone Angular app (Angular 17+), you pass this array to `provideRouter()` in `main.ts`.

```
// app.routes.ts — define your route configuration
import { Routes } from '@angular/router';
import { HomeComponent } from '../home/home.component';
import { AboutComponent } from '../about/about.component';
import { NotFoundComponent } from '../not-found/not-found.component';

export const APP_ROUTES: Routes = [
  { path: '', component: HomeComponent },           // default route
  { path: 'about', component: AboutComponent },
  { path: 'products', loadComponent: () =>          // lazy loaded
    import('../products/product-list.component')
    .then(m => m.ProductListComponent) },
  { path: '**', component: NotFoundComponent },     // wildcard 404
];

// main.ts — register the router
bootstrapApplication(AppComponent, {
```

```
providers: [provideRouter(APP_ROUTES)]
});
```

```
// app.component.ts - router-outlet renders the matched component
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav>
      <a routerLink="/" routerLinkActive='active'
[routerLinkActiveOptions]='{exact:true}'>Home</a>
      <a routerLink="/about" routerLinkActive='active'>About</a>
      <a routerLink="/products" routerLinkActive='active'>Products</a>
    </nav>
    <router-outlet></router-outlet>
  `
})
export class AppComponent {}
```

3. Route Parameters & Query Params

Routes often carry data in the URL — a product ID, a user slug, or filter options. Angular supports route parameters (part of the path) and query parameters (key=value pairs after the ?).

3.1 Route Parameters — :id

```
// app.routes.ts
{ path: 'products/:id', component: ProductDetailComponent }

// product-detail.component.ts
import { Component, inject, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

export class ProductDetailComponent implements OnInit {
  private route = inject(ActivatedRoute);
  productId!: string;

  ngOnInit() {
    // Snapshot: read once on component init
    this.productId = this.route.snapshot.paramMap.get('id')!;

    // Observable: reacts when param changes without destroying the component
    this.route.paramMap.subscribe(params => {
      this.productId = params.get('id')!;
      this.loadProduct(this.productId);
    });
  }

  loadProduct(id: string) { /* call ProductService.getById(id) */ }
```

```
}
```

3.2 Query Parameters — ?sort=price&page=2

```
export class ProductListComponent {
  private router = inject(Router);
  private route = inject(ActivatedRoute);

  applyFilter(sort: string, page: number) {
    this.router.navigate(['/products'], {
      queryParams: { sort, page },
      queryParamsHandling: 'merge' // preserve other existing params
    });
  }

  ngOnInit() {
    // Subscribe to query param changes reactively
    this.route.queryParamMap.subscribe(params => {
      const sort = params.get('sort') ?? 'name';
      const page = Number(params.get('page') ?? 1);
      this.loadProducts(sort, page);
    });
  }
}

<!-- Navigate with query params from the template -->
<a [routerLink]="['/products']" [queryParams]="{ sort: 'price', page: 1 }">
  Sort by Price
</a>
```

4. Nested (Child) Routes

Child routes render a component inside a parent component's own `<router-outlet>`. Essential for feature areas with shared layouts — a dashboard with a sidebar, or a settings page with tabs.

```
// app.routes.ts
export const APP_ROUTES: Routes = [
  {
    path: 'dashboard',
    component: DashboardComponent, // has its own <router-outlet>
    children: [
      { path: '', redirectTo: 'overview', pathMatch: 'full' },
      { path: 'overview', component: OverviewComponent },
      { path: 'analytics', component: AnalyticsComponent },
      { path: 'settings', component: SettingsComponent },
    ]
  }
];

// dashboard.component.html - shared layout with its own outlet
```

```
<aside class='sidebar'>
  <a routerLink='overview' routerLinkActive='active'>Overview</a>
  <a routerLink='analytics' routerLinkActive='active'>Analytics</a>
  <a routerLink='settings' routerLinkActive='active'>Settings</a>
</aside>
<main>
  <router-outlet></router-outlet>  <!-- children render here -->
</main>

// URL: /dashboard/analytics -> DashboardComponent wraps AnalyticsComponent
```

5. Lazy Loading

Lazy loading defers downloading a route's JavaScript bundle until the user navigates to it. This dramatically reduces the initial bundle size and speeds up first page load.

5.1 loadComponent — Lazy Load a Single Component

```
// app.routes.ts
export const APP_ROUTES: Routes = [
  { path: 'home', component: HomeComponent },           // eager
  {
    path: 'admin',
    loadComponent: () =>                               // lazy
      import('./admin/admin.component')
        .then(m => m.AdminComponent)
  },
  {
    path: 'reports',
    loadComponent: () => import('./reports/reports.component')
      .then(m => m.ReportsComponent)
  }
];
```

5.2 loadChildren — Lazy Load an Entire Feature

```
// app.routes.ts - loads the entire shop feature lazily
export const APP_ROUTES: Routes = [
  {
    path: 'shop',
    loadChildren: () =>
      import('./shop/shop.routes').then(m => m.SHOP_ROUTES)
  }
];

// shop/shop.routes.ts - the feature's own route config
export const SHOP_ROUTES: Routes = [
  { path: '', component: ShopHomeComponent },
  { path: 'cart', component: CartComponent },
  { path: 'checkout', component: CheckoutComponent },
```

```
{ path: ':id',      component: ProductDetailComponent },
];

// Angular creates a separate JS chunk for /shop — only downloaded on first visit
```

Lazy loading is one of the most impactful Angular performance wins. Users who never visit /admin never download that bundle. Split large feature areas aggressively with loadChildren.

6. Route Guards — Protecting Routes

Route guards are functions that run before a navigation event completes. They return true to allow, false to block, or a `UrlTree` to redirect. Modern Angular uses functional guards — plain functions, no classes needed.

6.1 canActivate — Auth Guard

```
// auth.guard.ts
import { inject }           from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService }       from '../auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) {
    return true;
  }

  // Redirect to /login and preserve the intended URL
  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url }
  });
};

// app.routes.ts — apply the guard
{ path: 'dashboard', component: DashboardComponent, canActivate: [authGuard] }

// canActivateChild — protects ALL children at once
{
  path: 'admin',
  component: AdminComponent,
  canActivateChild: [authGuard],
  children: [
    { path: 'users', component: UserManagementComponent },
    { path: 'settings', component: AdminSettingsComponent },
  ]
}
```

6.2 canDeactivate — Unsaved Changes Warning

```
// unsaved-changes.guard.ts
import { CanDeactivateFn } from '@angular/router';

export interface CanComponentDeactivate {
  hasUnsavedChanges(): boolean;
}

export const unsavedChangesGuard: CanDeactivateFn<CanComponentDeactivate> =
  (component) => {
    if (component.hasUnsavedChanges()) {
      return confirm('You have unsaved changes. Leave anyway?');
    }
    return true;
  };

// edit-form.component.ts
export class EditFormComponent implements CanComponentDeactivate {
  isDirty = false;
  hasUnsavedChanges() { return this.isDirty; }
}

{ path: 'edit/:id', component: EditFormComponent, canDeactivate:
  [unsavedChangesGuard] }
```

7. Route Resolvers — Pre-fetching Data

A resolver pre-fetches data before the route component is activated. The component receives ready-to-use data via `ActivatedRoute` rather than showing a loading spinner in `ngOnInit`.

```
// product.resolver.ts
import { inject } from '@angular/core';
import { ResolveFn } from '@angular/router';
import { ProductService } from '../product.service';
import { Product } from '../product.model';

export const productResolver: ResolveFn<Product> = (route) => {
  const id = Number(route.paramMap.get('id'));
  return inject(ProductService).getById(id); // Observable<Product>
};

// app.routes.ts
{
  path: 'products/:id',
  component: ProductDetailComponent,
  resolve: { product: productResolver }
}

// product-detail.component.ts - data is already loaded, no spinner needed
export class ProductDetailComponent implements OnInit {
  private route = inject(ActivatedRoute);
```

```
product!: Product;

ngOnInit() {
  // Data was pre-fetched - available immediately
  this.product = this.route.snapshot.data['product'];

  // Or subscribe to handle re-navigation to the same route
  this.route.data.subscribe(data => this.product = data['product']);
}
}
```

8. Programmatic Navigation with Router

Beyond template links, use the Router service to navigate from component code — after a login, form submission, or any conditional redirect logic.

```
import { Router, ActivatedRoute } from '@angular/router';
import { Location }                from '@angular/common';

export class LoginComponent {
  private router    = inject(Router);
  private route     = inject(ActivatedRoute);
  private location  = inject(Location);

  onLoginSuccess() {
    // Navigate to a fixed path
    this.router.navigate(['/dashboard']);

    // Navigate with a route param
    this.router.navigate(['/products', 42]);

    // Navigate with query params
    this.router.navigate(['/search'], { queryParams: { q: 'angular' } });

    // Navigate relative to current route
    this.router.navigate(['../sibling'], { relativeTo: this.route });

    // Replace current history entry (no back button entry added)
    this.router.navigateByUrl('/home', { replaceUrl: true });
  }

  goBack() {
    this.location.back(); // browser back button behaviour
  }
}
```

Interview Questions & Answers

Q1: What is the Angular Router and how does it work in a SPA?

- The Angular Router maps URL paths to components and manages navigation without full page reloads — this is what makes Angular a Single-Page Application
- You define a Routes array, provide it via `provideRouter()`, and place `<router-outlet>` where matched components should render
- When the user clicks a `routerLink` or the app calls `router.navigate()`, the Router updates the URL and renders the correct component with no page reload
- The Router manages the browser history stack via the `pushState` API so back/forward buttons work correctly

Q2: What is the difference between `router.navigate()` and `router.navigateByUrl()`?

- `router.navigate(['/path', param])` accepts a path segment array — supports relative navigation via the `relativeTo` option, easier to compose dynamic paths
- `router.navigateByUrl('/full/url')` accepts a URL string — always navigates absolutely, cannot use `relativeTo`
- `navigate()` is preferred for most use cases: `router.navigate(['/products', productId, 'reviews'])` composes paths cleanly
- Both accept `NavigationExtras` for `queryParams`, `replaceUrl`, `skipLocationChange`, and `state` options

Q3: What is lazy loading and why is it important?

- Lazy loading defers downloading a route's JavaScript bundle until the user navigates there for the first time
- It reduces the initial bundle size — the browser only downloads code for the current page, making app startup faster for all users
- Use `loadComponent` for a single standalone component or `loadChildren` for an entire feature route set — both use `dynamic import()`
- Angular's build tool automatically creates separate JS chunk files at every lazy boundary — users only download what they actually visit

Q4: What is a route guard and what guard types does Angular support?

- A route guard runs before a navigation completes and returns `true` to allow, `false` to block, or a `UrlTree` to redirect
- `canActivate` — blocks access to a route; used for authentication and authorisation checks
- `canActivateChild` — same as `canActivate` but applied to all children of a parent route at once
- `canDeactivate` — runs when leaving a route; warns users about unsaved changes before navigating away
- `canMatch` — prevents a route from matching at all; useful for feature flags or role-based routing

Q5: What is the difference between route params and query params?

- Route params are part of the URL path defined with a colon (products/:id) — they identify a specific resource and are required
- Query params appear after ? in the URL (?sort=price&page=2) — they are optional and represent view state like filters and pagination
- Read route params via route.snapshot paramMap.get('id') or the observable route.paramMap for reactive updates
- Use queryParamsHandling: 'merge' when navigating to update only specific query params without removing others

Q6: What is a route resolver and when would you use one?

- A resolver pre-fetches data before the route component activates — the component gets ready data instead of handling a loading state itself
- Without a resolver: component renders empty while fetching in ngOnInit — can cause a flash of empty content
- With a resolver: Angular waits for the Observable to complete, then renders the component with real data from the start
- Use resolvers when a page cannot be rendered meaningfully without its data, like a product detail or pre-populated edit form

Q7: What are child routes and when do you need them?

- Child routes render inside a parent component's own <router-outlet>, creating nested view hierarchies
- Use them when a section of your app has a shared layout (sidebar, tabs) with multiple swappable sub-views
- The parent component must include <router-outlet> in its own template — without it child components will never render
- Example: /dashboard with a sidebar and children /dashboard/overview and /dashboard/analytics sharing that sidebar layout

Q8: What is routerLinkActive and how does exact matching work?

- routerLinkActive automatically adds a CSS class to a nav element when its linked route is currently active — zero component logic needed
- Default is partial matching — /products link is active even when the URL is /products/123
- Add [routerLinkActiveOptions]='{ exact: true }' for exact URL matching — the class is only added when the URL matches precisely
- Can be placed on the <a> itself or on a parent — the class is applied to whichever element carries the routerLinkActive directive

Q9: What is the difference between pathMatch: 'full' and pathMatch: 'prefix'?

- pathMatch: 'prefix' (default) — route matches if the URL starts with the given path; empty string '' matches every URL

- `pathMatch: 'full'` — the entire URL must equal the path exactly; required on redirect routes with empty path to avoid infinite loops
- `{ path: '', redirectTo: '/home', pathMatch: 'full' }` — only redirects the exact root URL /
- Without 'full', the empty path would match all URLs and redirect everything to /home, breaking the app

Q10: How do you pass data to a route without using URL parameters?

- Static route data: `{ path: 'admin', data: { title: 'Admin', role: 'admin' } }` — read via `route.snapshot.data`; baked into the config at build time
- Router state: `router.navigate(['/detail'], { state: { product } })` — available via `history.state`; lost on page refresh
- Route resolver: pre-fetches and merges dynamic data into `route.data` before the component activates
- Shared singleton service: inject a service that holds state across navigations — most reliable for multi-step flows like checkout wizards

HTTP Client

Communicate with REST APIs, intercept requests, handle errors, and type your responses

1. Setting Up HttpClient

Angular's `HttpClient` is a powerful, built-in service for making HTTP requests. It is based on RxJS Observables — every request returns an Observable that you subscribe to or pipe through operators. Before using it, you must register the HTTP infrastructure with `provideHttpClient()` in `main.ts`.

```
// main.ts - standalone app setup
import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient, withInterceptorsFromDi } from '@angular/common/http';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient( // registers HttpClient globally
      withInterceptorsFromDi() // enables class-based interceptors (optional)
    )
  ]
});

// Inject HttpClient into any service or component
import { HttpClient } from '@angular/common/http';
import { inject } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
}
```

Always use `HttpClient` inside a service — never directly inside a component. Services centralise API logic, making it reusable and independently testable without touching the component.

2. Making HTTP Requests — GET, POST, PUT, PATCH, DELETE

HttpClient provides a method for every HTTP verb. Each method is generic — you pass a TypeScript interface as the type parameter and HttpClient will cast the response to that type automatically.

2.1 GET — Fetching Data

```
// models/product.model.ts
export interface Product {
  id:          number;
  name:        string;
  price:       number;
  category:    string;
  inStock:     boolean;
}

// product.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpParams } from '@angular/common/http';
import { Observable }           from 'rxjs';
import { Product }              from '../models/product.model';

@Injectable({ providedIn: 'root' })
export class ProductService {
  private http    = inject(HttpClient);
  private baseUrl = 'https://api.example.com/products';

  // GET all products - returns Observable<Product[]>
  getAll(): Observable<Product[]> {
    return this.http.get<Product[]>(this.baseUrl);
  }

  // GET with query parameters - /products?category=electronics&inStock=true
  getFiltered(category: string, inStock: boolean): Observable<Product[]> {
    const params = new HttpParams()
      .set('category', category)
      .set('inStock', String(inStock));
    return this.http.get<Product[]>(this.baseUrl, { params });
  }

  // GET single resource - /products/42
  getById(id: number): Observable<Product> {
    return this.http.get<Product>(`${this.baseUrl}/${id}`);
  }
}
```

2.2 POST — Creating Resources

```
// POST - send data to create a new resource
createProduct(product: Omit<Product, 'id'>): Observable<Product> {
  return this.http.post<Product>(this.baseUrl, product);
}

// POST with custom headers
```

```
import { HttpHeaders } from '@angular/common/http';

createWithAuth(product: Omit<Product, 'id'>, token: string): Observable<Product> {
  const headers = new HttpHeaders({
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`
  });
  return this.http.post<Product>(this.baseUrl, product, { headers });
}

// POST form data (file upload)
uploadImage(file: File): Observable<{ url: string }> {
  const formData = new FormData();
  formData.append('image', file, file.name);
  return this.http.post<{ url: string }>('/api/upload', formData);
}
```

2.3 PUT, PATCH, DELETE

```
// PUT - replace entire resource
updateProduct(id: number, product: Product): Observable<Product> {
  return this.http.put<Product>(`${this.baseUrl}/${id}`, product);
}

// PATCH - update only specific fields
patchProduct(id: number, changes: Partial<Product>): Observable<Product> {
  return this.http.patch<Product>(`${this.baseUrl}/${id}`, changes);
}

// DELETE - returns void (no response body expected)
deleteProduct(id: number): Observable<void> {
  return this.http.delete<void>(`${this.baseUrl}/${id}`);
}

// DELETE with body (rare but sometimes needed)
bulkDelete(ids: number[]): Observable<void> {
  return this.http.delete<void>(this.baseUrl, { body: { ids } });
}
```

3. Consuming HTTP Observables in Components

Components subscribe to the service Observables. The recommended approach is to use the async pipe in the template — it subscribes automatically and cleans up when the component is destroyed, preventing memory leaks.

3.1 Using the async Pipe (Recommended)

```
// product-list.component.ts
import { Component, inject, OnInit } from '@angular/core';
```

```
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import { ProductService, Product } from '../product.service';
import { Observable } from 'rxjs';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [AsyncPipe, CurrencyPipe],
  template: `
    @if (products$ | async; as products) {
      @for (p of products; track p.id) {
        <div class='card'>
          <h3>{{ p.name }}</h3>
          <span>{{ p.price | currency }}</span>
        </div>
      }
    } @else {
      <p>Loading products...</p>
    }
  `
})
export class ProductListComponent implements OnInit {
  private productService = inject(ProductService);
  products$: Observable<Product[]>;

  ngOnInit() {
    this.products$ = this.productService.getAll();
  }
}
```

3.2 Manual Subscribe with Cleanup

```
// When you need the data in the class (not just the template)
import { Component, inject, OnDestroy, OnInit } from '@angular/core';
import { Subscription } from 'rxjs';

export class ProductListComponent implements OnInit, OnDestroy {
  private productService = inject(ProductService);
  private sub!: Subscription;
  products: Product[] = [];
  isLoading = true;
  error: string | null = null;

  ngOnInit() {
    this.sub = this.productService.getAll().subscribe({
      next: (data) => { this.products = data; this.isLoading = false; },
      error: (err) => { this.error = err.message; this.isLoading = false; },
      complete: () => { console.log('Request complete'); }
    });
  }

  ngOnDestroy() {
    this.sub.unsubscribe(); //  prevent memory leaks
  }
}
```

```
}
```

4. Error Handling

HTTP requests can fail due to network issues, server errors, or bad requests. Angular's `HttpClient` surfaces these as RxJS errors in the `Observable` stream. Handle them in the service using `catchError`, and expose friendly messages to the component.

```
import { HttpClient, HttpResponseError } from '@angular/common/http';
import { Observable, throwError }         from 'rxjs';
import { catchError, retry }              from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class ProductService {
  private http = inject(HttpClient);

  getAll(): Observable<Product[]> {
    return this.http.get<Product[]>(this.baseUrl).pipe(
      retry(2), // retry failed requests up to 2 times
      catchError(this.handleError) // then handle the error
    );
  }

  private handleError(error: HttpResponseError): Observable<never> {
    let message = 'An unknown error occurred';

    if (error.status === 0) {
      // Network error (no internet, CORS, etc.)
      message = 'Network error – please check your connection';
    } else {
      // Server returned an error status (4xx, 5xx)
      switch (error.status) {
        case 400: message = 'Bad request – invalid data sent'; break;
        case 401: message = 'Unauthorised – please log in again'; break;
        case 403: message = 'Forbidden – insufficient permissions'; break;
        case 404: message = 'Resource not found'; break;
        case 500: message = 'Server error – please try again later'; break;
        default: message = `Unexpected error: ${error.status}`;
      }
    }

    console.error('HTTP Error:', error);
    return throwError(() => new Error(message));
  }
}
```

5. HTTP Interceptors

HTTP Interceptors are middleware functions that sit between your application code and the actual HTTP request/response. They let you add authentication headers, log requests, handle errors globally, or show a loading spinner — without modifying every individual service call.

5.1 Functional Interceptor (Modern — Angular 15+)

```
// auth.interceptor.ts - add JWT token to every request
import { HttpInterceptorFn } from '@angular/common/http';
import { inject }           from '@angular/core';
import { AuthService }      from '../auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.getToken();

  if (!token) {
    return next(req);          // no token - pass request unchanged
  }

  // Clone the request and add the Authorization header
  const authReq = req.clone({
    headers: { Authorization: `Bearer ${token}` }
  });

  return next(authReq);        // forward the modified request
};

// main.ts - register the interceptor
import { provideHttpClient, withInterceptors } from '@angular/common/http';

bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
});
```

5.2 Logging + Error Interceptor

```
// logging.interceptor.ts - log all requests and handle 401 globally
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject }           from '@angular/core';
import { catchError } from 'rxjs/operators';
import { throwError } from 'rxjs';
import { Router }         from '@angular/router';

export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  const start = Date.now();

  console.log(`--> ${req.method} ${req.url}`);

  return next(req).pipe(
```

```

catchError((err: HttpErrorResponse) => {
  const elapsed = Date.now() - start;
  console.error(`<-- ${req.method} ${req.url} ${err.status} (${elapsed}ms)`);

  // Globally redirect to /login on 401 Unauthorized
  if (err.status === 401) {
    router.navigate(['/login']);
  }

  return throwError(() => err);
})
);
};

// Register multiple interceptors - they run in order
provideHttpClient(withInterceptors([authInterceptor, loggingInterceptor]))

```

5.3 Loading Spinner Interceptor

```

// loading.interceptor.ts - show/hide a global loading spinner
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { LoadingService } from '../loading.service';
import { finalize } from 'rxjs/operators';

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loader = inject(LoadingService);
  loader.show();

  return next(req).pipe(
    finalize(() => loader.hide()) // always hide, even on error
  );
};

// loading.service.ts
@Injectable({ providedIn: 'root' })
export class LoadingService {
  private count = 0;
  isLoading$ = new BehaviorSubject<boolean>(false);

  show() { if (++this.count > 0) this.isLoading$.next(true); }
  hide() { if (--this.count <= 0) this.isLoading$.next(false); }
}

```

Interceptors run in the order they are registered. Put `authInterceptor` first (to attach the token), then `loggingInterceptor` (to log the final request including auth headers), then `loadingInterceptor` last.

6. Advanced HttpClient — Full Response, Progress Events, Caching

6.1 Accessing the Full HTTP Response

By default, HttpClient returns only the response body. Pass `observe: 'response'` to access status codes, headers, and the full `HttpResponse` object.

```
// Get the full response including status and headers
import { HttpResponse } from '@angular/common/http';

getWithHeaders(): Observable<HttpResponse<Product[]>> {
  return this.http.get<Product[]>(this.baseUrl, { observe: 'response' });
}

// Usage
this.productService.getWithHeaders().subscribe(response => {
  console.log('Status:', response.status); // 200
  console.log('X-Total:', response.headers.get('X-Total-Count'));
  console.log('Body:', response.body); // Product[]
});
```

6.2 Upload Progress Tracking

```
import { HttpEventType, HttpRequest } from '@angular/common/http';

uploadFile(file: File): Observable<number> {
  const formData = new FormData();
  formData.append('file', file);

  const req = new HttpRequest('POST', '/api/upload', formData, {
    reportProgress: true // enables progress events
  });

  return new Observable(observer => {
    this.http.request(req).subscribe({
      next: event => {
        if (event.type === HttpEventType.UploadProgress && event.total) {
          const percent = Math.round(100 * event.loaded / event.total);
          observer.next(percent); // emit 0..100
        } else if (event.type === HttpEventType.Response) {
          observer.complete();
        }
      },
      error: err => observer.error(err)
    });
  });
}

// Template
<progress [value]='uploadProgress' max='100'></progress>
<span>{{ uploadProgress }}%</span>
```

6.3 Simple In-Service Caching with shareReplay

```
import { shareReplay } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class CategoryService {
  private http = inject(HttpClient);

  // Cache the result - categories rarely change
  // shareReplay(1) replays the last emission to new subscribers
  // without making a new HTTP request
  private categories$ = this.http
    .get<string[]>('/api/categories')
    .pipe(shareReplay(1));

  getCategories(): Observable<string[]> {
    return this.categories$; // same Observable reused for all subscribers
  }

  // Invalidate the cache when needed
  refreshCategories(): void {
    this.categories$ = this.http
      .get<string[]>('/api/categories')
      .pipe(shareReplay(1));
  }
}
```

7. Combining HTTP Requests with RxJS

Since `HttpClient` returns Observables, you can compose multiple HTTP requests using RxJS operators. This is far cleaner and more powerful than chaining Promises.

```
import { forkJoin, switchMap, combineLatest } from 'rxjs';

// forkJoin - run multiple requests IN PARALLEL, wait for all to complete
loadDashboard(): Observable<{ products: Product[], categories: string[] }> {
  return forkJoin({
    products: this.http.get<Product[]>('/api/products'),
    categories: this.http.get<string[]>('/api/categories')
  });
}

// switchMap - chain requests sequentially (result of first feeds into second)
// Cancels the inner request if the outer emits again (prevents race conditions)
getProductWithReviews(id: number) {
  return this.http.get<Product>(`/api/products/${id}`).pipe(
    switchMap(product =>
      this.http.get<Review[]>(`/api/reviews?productId=${product.id}`).pipe(
        map(reviews => ({ ...product, reviews })))
    )
}
```

```
    )
  );
}

// combineLatest - react every time any source emits (good for filters +
// pagination)
loadFilteredProducts(
  category$: Observable<string>,
  page$:      Observable<number>
) {
  return combineLatest([category$, page$]).pipe(
    switchMap(([category, page]) =>
      this.http.get<Product[]>('/api/products', {
        params: new HttpParams().set('category', category).set('page',
String(page))
      })
    )
  );
}
```

Interview Questions & Answers

Q1: What is HttpClient and how do you set it up in a standalone Angular app?

- HttpClient is Angular's built-in HTTP service that wraps the browser's XMLHttpRequest and fetch APIs, returning RxJS Observables for every request
- In a standalone app, register it in main.ts with provideHttpClient() inside the providers array of bootstrapApplication()
- To use functional interceptors, pass withInterceptors([...]) to provideHttpClient(); for class-based interceptors, use withInterceptorsFromDi()
- Inject HttpClient into services (not components directly) using inject(HttpClient) or constructor injection

Q2: What is the difference between HttpClient.get() and a raw fetch() call?

- HttpClient returns RxJS Observables — they are lazy (no request fires until you subscribe), cancellable (unsubscribe cancels the request), and composable with operators
- Raw fetch() returns a Promise — eager (fires immediately), not cancellable without an AbortController, and harder to compose
- HttpClient automatically parses JSON responses, handles error status codes as stream errors, and integrates with Angular's zone-based change detection
- Interceptors are exclusive to HttpClient — there is no equivalent middleware layer for raw fetch() in Angular

Q3: What is an HTTP Interceptor and what can you use it for?

- An interceptor is a middleware function that intercepts every outgoing request and every incoming response before they reach your service code
- Common uses: attaching JWT Authorization headers to every request, logging request/response timing, showing a global loading spinner, and redirecting to /login on 401 errors
- Modern Angular (15+) uses functional interceptors: `HttpInterceptorFn = (req, next) => next(req.clone({...}))` — no class needed
- Multiple interceptors are registered with `withInterceptors([a, b, c])` and run in array order for requests; in reverse order for responses

Q4: Why should you clone the request in an interceptor instead of mutating it?

- `HttpRequest` objects are immutable by design — Angular prevents you from mutating them directly to avoid subtle bugs from shared state
- `req.clone({ setHeaders: { Authorization: token } })` creates a new request object with only the specified properties changed; all other properties are preserved
- This immutability ensures interceptors are pure functions — they do not have side effects on the original request object
- The clone method supports `setHeaders`, `setParams`, `body`, `url`, `method`, and `withCredentials` among others

Q5: What is the difference between `observe: 'body'` and `observe: 'response'`?

- `observe: 'body'` (default) — `HttpClient` unwraps the response and returns only the parsed body; the type is `T` from `http.get<T>()`
- `observe: 'response'` — `HttpClient` returns the full `HttpResponse<T>` object including status code, status text, and all response headers
- Use `observe: 'response'` when you need to read custom headers (e.g. `X-Total-Count` for pagination), check the exact status code, or process both headers and body together
- A third option, `observe: 'events'`, streams all `HttpEvent` types including `UploadProgress`, `DownloadProgress`, `Sent`, and `Response` — needed for file upload progress bars

Q6: How do you handle HTTP errors in Angular?

- Use the `catchError` RxJS operator inside the service's `pipe()` to intercept errors before they reach the component
- The error is an `HttpErrorResponse` object: check `error.status` for the HTTP code (404, 500) and `error.status === 0` for network/CORS errors
- Return `throwError(() => new Error(friendlyMessage))` from `catchError` to propagate a cleaned-up error to the subscriber
- Use `retry(n)` before `catchError` to automatically re-attempt failed requests a specified number of times before giving up

Q7: What is the difference between `switchMap`, `mergeMap`, and `concatMap` when chaining HTTP requests?

- `switchMap` — cancels the previous inner Observable when a new outer value arrives; ideal for search-as-you-type or navigation — prevents stale responses from arriving out of order
- `concatMap` — queues inner Observables and waits for each to complete before starting the next; ideal for sequential dependent operations where order matters
- `mergeMap` (`flatMap`) — runs all inner Observables concurrently with no cancellation; use when order does not matter and you want maximum parallelism
- For HTTP, `switchMap` is usually correct for user-driven searches; `concatMap` for form submissions that must not be reordered; `forkJoin` for truly parallel independent requests

Q8: How do you add query parameters to an HTTP request using `HttpParams`?

- Create a new `HttpParams()` instance and chain `.set(key, value)` calls — `HttpParams` is also immutable, so each `.set()` returns a new instance
- Pass it as the `params` property in the options object: `this.http.get<T>(url, { params })`
- To build params dynamically from an object, use: `new HttpParams({ fromObject: { sort: 'name', page: '1' } })`
- For array values (e.g. multiple filter tags), use `params.append('tag', 'angular').append('tag', 'rxjs')` — this creates `tag=angular&tag=rxjs` in the URL

Q9: What is `shareReplay` and how does it help with caching HTTP responses?

- `shareReplay(1)` is an RxJS operator that multicasts an Observable and replays the last N emissions to new subscribers — without re-executing the source
- When applied to an HTTP Observable, the first subscriber triggers the HTTP request; subsequent subscribers receive the cached response immediately with no new request
- It is ideal for reference data that rarely changes: categories, countries, config — fetched once and shared across the app
- To invalidate the cache, reassign the `cached$` Observable to a fresh `http.get().pipe(shareReplay(1))` call

Q10: What is the difference between `forkJoin` and `combineLatest` for parallel HTTP requests?

- `forkJoin` waits for ALL source Observables to complete and then emits a single combined result — perfect for fire-and-forget parallel HTTP calls that each complete after one emission
- `combineLatest` emits whenever any source emits, using the latest values from all sources — suited for reactive streams like filter dropdowns that change over time
- HTTP Observables complete after one emission, so `forkJoin` is the correct choice for parallel API calls: `forkJoin({ users, products, categories })` fires all three simultaneously and waits for all
- `combineLatest` is better when combining an HTTP call with an ongoing stream like a search Subject, so results update every time the search term changes

Forms

Template-driven and reactive forms — building, validating, and submitting user input in Angular

1. Two Approaches to Forms in Angular

Angular provides two distinct approaches to building forms: Template-Driven Forms and Reactive Forms. Both capture user input, validate it, and give you access to the form's state — but they differ in where the form model lives and how much control you have.

```
// TEMPLATE-DRIVEN FORMS
// - Form model lives in the template (HTML-first)
// - Uses ngModel for two-way binding
// - Simple to set up, good for basic forms
// - Less testable — logic is tied to the template
// - Import: FormsModule

// REACTIVE FORMS
// - Form model lives in the component class (TypeScript-first)
// - Uses FormControl, FormGroup, FormArray
// - More verbose but fully explicit and testable
// - Better for dynamic forms and complex validation
// - Import: ReactiveFormsModule
```

For any form beyond a simple contact form, prefer Reactive Forms. They give you full programmatic control, easier unit testing, better TypeScript support, and cleaner validation logic — especially for dynamic forms.

2. Template-Driven Forms

Template-driven forms rely on Angular directives in the HTML template to build and track the form model. The `NgForm` and `NgModel` directives automatically create a `FormGroup` and `FormControl` behind the scenes. You access the form state via template reference variables.

2.1 Basic Template-Driven Form

```
// login.component.ts
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
```

```

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [FormsModule],
  template: `
    <form #loginForm='ngForm' (ngSubmit)='onSubmit(loginForm)'>

      <label>Email
        <input
          type='email'
          name='email'
          [(ngModel)]='model.email'
          #emailField='ngModel'
          required
          email>
        @if (emailField.invalid && emailField.touched) {
          <span class='error'>Valid email is required</span>
        }
      </label>

      <label>Password
        <input
          type='password'
          name='password'
          [(ngModel)]='model.password'
          #pwField='ngModel'
          required
          minlength='8'>
        @if (pwField.errors?.['minlength'] && pwField.touched) {
          <span class='error'>Password must be at least 8 characters</span>
        }
      </label>

      <button type='submit' [disabled]='loginForm.invalid'>Log In</button>
    </form>
  `,
})
export class LoginComponent {
  model = { email: '', password: '' };

  onSubmit(form: NgForm) {
    if (form.valid) {
      console.log('Form data:', form.value);
      form.reset();
    }
  }
}

```

2.2 Template-Driven Form State Classes

Angular automatically adds CSS classes to form controls based on their state. You can use these to style inputs conditionally:

```
// Angular adds these CSS classes automatically to inputs with ngModel:
```

```
// ng-untouched / ng-touched      - user has not/has interacted with the field
// ng-pristine / ng-dirty         - value has not/has been changed
// ng-valid / ng-invalid          - passes/fails validation

/* styles.scss - style invalid touched inputs with a red border */
input.ng-invalid.ng-touched {
  border: 2px solid red;
}

input.ng-valid.ng-touched {
  border: 2px solid green;
}
```

3. Reactive Forms — FormControl, FormGroup, FormArray

Reactive Forms are built entirely in the component class using `FormControl`, `FormGroup`, and `FormArray`. The template only binds to this pre-built model using the `formGroup`, `formControlName`, and `formArrayName` directives.

3.1 FormControl — Single Field

```
import { Component }           from '@angular/core';
import { ReactiveFormsModule, FormControl,
       Validators }           from '@angular/forms';

@Component({
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <input [formControl]='emailCtrl' type='email' placeholder='Email'>

    @if (emailCtrl.invalid && emailCtrl.touched) {
      @if (emailCtrl.errors?.['required']) {
        <p class='error'>Email is required</p>
      }
      @if (emailCtrl.errors?.['email']) {
        <p class='error'>Enter a valid email address</p>
      }
    }

    <p>Status: {{ emailCtrl.status }}</p>    <!-- VALID | INVALID | PENDING -->
    <p>Value:  {{ emailCtrl.value }}</p>
  `
})
export class EmailComponent {
  emailCtrl = new FormControl('', [Validators.required, Validators.email]);

  // Subscribe to value changes (useful for search-as-you-type)
  constructor() {
    this.emailCtrl.valueChanges.subscribe(val => console.log(val));
  }
}
```

```
}

```

3.2 FormGroup — Grouped Fields

```
import { Component }           from '@angular/core';
import { ReactiveFormsModule,
       FormGroup, FormControl,
       Validators }           from '@angular/forms';

@Component({
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]='profileForm' (ngSubmit)='onSubmit()'>

      <input formControlName='firstName' placeholder='First Name'>
      @if (f['firstName'].invalid && f['firstName'].touched) {
        <p class='error'>First name is required</p>
      }

      <input formControlName='lastName' placeholder='Last Name'>

      <!-- Nested FormGroup -->
      <div formGroupName='address'>
        <input formControlName='street' placeholder='Street'>
        <input formControlName='city' placeholder='City'>
        <input formControlName='pincode' placeholder='Pincode'>
      </div>

      <button type='submit' [disabled]='profileForm.invalid'>Save</button>
    </form>
  `
})
export class ProfileComponent {
  profileForm = new FormGroup({
    firstName: new FormControl('', [Validators.required,
    Validators.minLength(2)]),
    lastName: new FormControl('', Validators.required),
    address: new FormGroup({
      street: new FormControl(''),
      city: new FormControl('', Validators.required),
      pincode: new FormControl('', Validators.pattern('^[0-9]{6}$')),
    })
  });

  // Shorthand getter for cleaner template access
  get f() { return this.profileForm.controls; }

  onSubmit() {
    if (this.profileForm.valid) {
      console.log(this.profileForm.value);
      // { firstName: 'Gowtham', lastName: '...', address: { ... } }
    }
  }
}
```

```
}
```

4. FormBuilder — Cleaner Reactive Form Syntax

FormBuilder is an Angular service that provides shorthand methods to create FormControl, FormGroup, and FormArray instances — removing repetitive new FormControl() boilerplate.

```
import { Component, inject }           from '@angular/core';
import { ReactiveFormsModule,
       FormBuilder, Validators }       from '@angular/forms';

@Component({ standalone: true, imports: [ReactiveFormsModule], template: `...` })
export class RegisterComponent {
  private fb = inject(FormBuilder);

  // FormBuilder.group() is shorthand for new FormGroup({...})
  registerForm = this.fb.group({
    name:          ['',      [Validators.required, Validators.minLength(3)]],
    email:         ['',      [Validators.required, Validators.email]],
    password:      ['',      [Validators.required, Validators.minLength(8)]],
    confirmPassword: ['',    Validators.required],
    role:          ['user',   Validators.required],
    agreeToTerms:  [false,    Validators.requiredTrue],
  }, {
    validators: passwordMatchValidator // cross-field validator (see Section 6)
  });

  get email()    { return this.registerForm.get('email')!; }
  get password() { return this.registerForm.get('password')!; }

  onSubmit() {
    if (this.registerForm.valid) {
      const { confirmPassword, ...payload } = this.registerForm.value;
      console.log('Register:', payload);
    } else {
      this.registerForm.markAllAsTouched(); // show all errors at once
    }
  }
}
```

5. FormArray — Dynamic Lists of Controls

FormArray holds an ordered list of FormControls, FormGroups, or other FormArrays. It is the correct tool for dynamic forms where the user can add or remove rows — like a list of phone numbers, skills, or addresses.

```
import { Component, inject }           from '@angular/core';
```

```

import { ReactiveFormsModule, FormBuilder,
        FormArray, Validators } from '@angular/forms';

@Component({
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]='form'>
      <h3>Skills</h3>

      <div formArrayName='skills'>
        @for (ctrl of skills.controls; track $index) {
          <div>
            <input [formControlName]='$index' placeholder='Skill'>
            <button type='button' (click)='removeSkill($index)'>Remove</button>
          </div>
        }
      </div>

      <button type='button' (click)='addSkill()'>+ Add Skill</button>
      <button type='submit' (click)='onSubmit()'>Submit</button>
    </form>
  `,
})
export class SkillsFormComponent {
  private fb = inject(FormBuilder);

  form = this.fb.group({
    skills: this.fb.array([
      this.fb.control('Angular', Validators.required),
      this.fb.control('TypeScript'),
    ])
  });

  // Getter for easy access in template and class
  get skills(): FormArray {
    return this.form.get('skills') as FormArray;
  }

  addSkill() {
    this.skills.push(this.fb.control('', Validators.required));
  }

  removeSkill(index: number) {
    this.skills.removeAt(index);
  }

  onSubmit() {
    console.log(this.form.value);
    // { skills: ['Angular', 'TypeScript', ...] }
  }
}

```

6. Validation — Built-in, Custom, and Cross-Field

Angular ships with several built-in validators and supports fully custom synchronous and asynchronous validators. Validators are plain functions that receive an `AbstractControl` and return either null (valid) or an error object (invalid).

6.1 Built-in Validators

```
import { Validators } from '@angular/forms';

this.fb.group({
  username: ['', [Validators.required, Validators.minLength(3),
Validators.maxLength(20)]],
  email:     ['', [Validators.required, Validators.email]],
  age:       ['', [Validators.required, Validators.min(18),
Validators.max(120)]],
  website:   ['', Validators.pattern('https?:/.+')],
  terms:     [false, Validators.requiredTrue],
  // Validators.nullValidator - always returns null (no-op placeholder)
});
```

6.2 Custom Synchronous Validator

```
// no-spaces.validator.ts
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

export function noSpacesValidator(): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const hasSpaces = (control.value as string)?.includes(' ');
    return hasSpaces ? { noSpaces: { value: control.value } } : null;
    // return null → valid
    // return object → invalid; key is the error name you check in template
  };
}

// Usage in FormBuilder
username: ['', [Validators.required, noSpacesValidator()]]

// Template - read the error
@if (f['username'].errors?.['noSpaces']) {
  <p class='error'>Username cannot contain spaces</p>
}
```

6.3 Cross-Field Validator (Password Match)

```
// password-match.validator.ts
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

export const passwordMatchValidator: ValidatorFn = (
  group: AbstractControl
): ValidationErrors | null => {
  const password = group.get('password')?.value;
```

```
const confirmPassword = group.get('confirmPassword')?.value;
return password === confirmPassword ? null : { passwordMismatch: true };
};

// Applied at the FormGroup level (not individual control)
registerForm = this.fb.group({
  password: ['', Validators.required],
  confirmPassword: ['', Validators.required],
}, { validators: passwordMatchValidator });

// Template - check the GROUP's errors (not the control's)
@if (registerForm.errors?.['passwordMismatch'] && registerForm.touched) {
  <p class='error'>Passwords do not match</p>
}
```

6.4 Async Validator — Check Availability via API

```
// username-taken.validator.ts
import { AbstractControl, AsyncValidatorFn, ValidationErrors } from
'@angular/forms';
import { Observable, of } from 'rxjs';
import { map, catchError, debounceTime, switchMap } from 'rxjs/operators';
import { UserService } from './user.service';
import { inject } from '@angular/core';

export function usernameTakenValidator(): AsyncValidatorFn {
  const userService = inject(UserService);

  return (control: AbstractControl): Observable<ValidationErrors | null> => {
    if (!control.value) return of(null);

    return of(control.value).pipe(
      debounceTime(400), // wait 400ms after user stops typing
      switchMap(username =>
        userService.checkUsername(username).pipe(
          map(isTaken => isTaken ? { usernameTaken: true } : null),
          catchError(() => of(null)) // on API error, treat as valid
        )
      )
    );
  };
}

// Usage - async validators go in the THIRD argument
username: ['', [Validators.required], [usernameTakenValidator()]]

// Template - control.status === 'PENDING' while async validator runs
@if (f['username'].status === 'PENDING') {
  <span>Checking availability...</span>
}
@if (f['username'].errors?.['usernameTaken']) {
  <p class='error'>Username is already taken</p>
}
```

7. Form Control Methods & State Management

Reactive Forms give you full programmatic control over every aspect of the form — setting values, resetting, patching, disabling controls, and subscribing to changes.

```
const form = this.fb.group({
  name: [''],
  email: [''],
  role: ['user'],
});

// — Setting values —————
// setValue: must provide ALL fields — throws error if any is missing
form.setValue({ name: 'Gowtham', email: 'g@example.com', role: 'admin' });

// patchValue: provide ONLY the fields you want to update — safe for partial updates
form.patchValue({ name: 'Gowtham' });

// Set a single control's value
form.get('email')?.setValue('gowtham@example.com');

// — Resetting —————
form.reset(); // clears all values and resets state
form.reset({ name: '', role: 'user' }); // reset to specific values
form.get('email')?.reset(); // reset a single control

// — Disabling/Enabling —————
form.get('role')?.disable(); // disable (excluded from form.value by default)
form.get('role')?.enable(); // re-enable
form.disable(); // disable entire form

// — State checks —————
form.valid; // true if all controls pass validation
form.invalid; // true if any control fails
form.dirty; // true if any value has changed
form.pristine; // true if no value has changed
form.touched; // true if any control was blurred
form.get('name')?.errors; // null or { required: true, minLength: {...} }

// — Marking state —————
form.markAllAsTouched(); // trigger validation display for ALL fields at once
form.markAsDirty();
form.markAsPristine();

// — Listening to changes —————
form.valueChanges.subscribe(val => console.log('Form value:', val));
form.statusChanges.subscribe(s => console.log('Status:', s)); //
VALID|INVALID|PENDING
form.get('name')?.valueChanges.subscribe(v => console.log('Name:', v));
```

8. Complete Reactive Form — Real-World Example

Here is a complete registration form combining FormBuilder, nested groups, custom validators, async validation, and a submit handler with error display:

```
// register.component.ts
import { Component, inject } from '@angular/core';
import { ReactiveFormsModule, FormBuilder,
       Validators, AbstractControl } from '@angular/forms';
import { noSpacesValidator } from './no-spaces.validator';
import { usernameTakenValidator } from './username-taken.validator';
import { passwordMatchValidator } from './password-match.validator';
import { AuthService } from './auth.service';
import { Router } from '@angular/router';

@Component({
  selector: 'app-register',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './register.component.html'
})
export class RegisterComponent {
  private fb = inject(FormBuilder);
  private auth = inject(AuthService);
  private router = inject(Router);

  isSubmitting = false;
  serverError = '';

  form = this.fb.group({
    username: ['', [Validators.required, Validators.minLength(3),
                    noSpacesValidator()],
               [usernameTakenValidator()]],
    email: ['', [Validators.required, Validators.email]],
    passwords: this.fb.group({
      password: ['', [Validators.required, Validators.minLength(8)]],
      confirmPassword: ['', Validators.required],
    }, { validators: passwordMatchValidator }),
  });

  // Getters for clean template access
  get username() { return this.form.get('username')!; }
  get email() { return this.form.get('email')!; }
  get passwords() { return this.form.get('passwords')!; }
  get password() { return this.passwords.get('password')!; }

  onSubmit() {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
      return;
    }

    this.isSubmitting = true;
    const { username, email, passwords } = this.form.value;

    this.auth.register({ username, email, password: passwords.password })
      .subscribe({
```

```
    next: () => this.router.navigate(['/dashboard']),
    error: (err) => { this.serverError = err.message; this.isSubmitting =
false; }
  });
}
```

Interview Questions & Answers

Q1: What is the difference between Template-Driven and Reactive Forms?

- Template-Driven Forms: the form model is created implicitly by directives in the HTML template (NgForm, NgModel); simple to set up but harder to test and less flexible
- Reactive Forms: the form model is built explicitly in the component class using FormControl, FormGroup, and FormArray; verbose but fully testable, type-safe, and better for dynamic or complex forms
- Template-Driven requires FormsModule; Reactive requires ReactiveFormsModule — both can be used in the same app for different forms
- For most production applications, Reactive Forms are preferred because the entire form structure is visible in the TypeScript class, not scattered across the HTML template

Q2: What are FormControl, FormGroup, and FormArray?

- FormControl — the most basic unit; tracks the value and validation state of a single form input (text, checkbox, select)
- FormGroup — a collection of FormControls with named keys; tracks the combined validity of all its child controls; maps to a single form or a nested section of a form
- FormArray — an ordered list of FormControls or FormGroups; used for dynamic inputs where the user can add or remove items at runtime (e.g., a list of skills or addresses)
- All three extend AbstractControl and share common properties: value, valid, invalid, dirty, pristine, touched, errors, valueChanges, and statusChanges

Q3: What is FormBuilder and why use it?

- FormBuilder is an Angular service that provides helper methods (group(), control(), array()) as shorthand for creating reactive form instances
- Without it: new FormGroup({ name: new FormControl("", Validators.required) }) — verbose and repetitive
- With it: this.fb.group({ name: ["", Validators.required] }) — much cleaner, especially for large forms with many fields
- Inject it with inject(FormBuilder) or constructor injection and use fb.group(), fb.control(), and fb.array() to build the form model

Q4: What is the difference between setValue() and patchValue()?

- `setValue()` requires you to provide a value for every single control in the `FormGroup` — it throws an error if any key is missing or extra
- `patchValue()` accepts a partial object — you only provide the fields you want to update; any fields not mentioned are left unchanged
- Use `setValue()` when you want to guarantee a complete reset to known values (e.g., loading a full entity from the server into an edit form)
- Use `patchValue()` for partial updates, for example when only one section of a large form needs to be pre-populated

Q5: How do you create a custom validator in Angular?

- A custom validator is a function that implements `ValidatorFn` — it receives an `AbstractControl` and returns null if valid or a `ValidationErrors` object if invalid
- To use it as a factory (to accept parameters): `export function minAgeValidator(min: number): ValidatorFn { return (control) => control.value >= min ? null : { minAge: { min } }; }`
- Apply it in the validators array: `fb.group({ age: ['', [Validators.required, minAgeValidator(18)]] })`
- Read the error in the template: `ctrl.errors?.['minAge']` gives you the error object you returned from the validator

Q6: What is a cross-field validator and how do you implement one?

- A cross-field validator is applied to a `FormGroup` (not a single control) so it can access and compare the values of multiple controls — e.g., verifying that password and confirmPassword match
- It is passed via the second argument to `FormGroup`/`fb.group()`: `this.fb.group({ ... }, { validators: passwordMatchValidator })`
- Inside the validator, use `group.get('password')?.value` and `group.get('confirmPassword')?.value` to access sibling controls
- The error is set on the group, not on individual controls — check `registerForm.errors?.['passwordMismatch']` in the template, not on the control

Q7: What is an async validator and when would you use one?

- An async validator is a function that implements `AsyncValidatorFn` — it returns an `Observable<ValidationErrors | null>` or `Promise` instead of a synchronous value
- Use cases: checking if a username or email is already taken via an API call before allowing the user to submit
- Async validators are passed as the third argument to `FormControl`: `fb.control('', [sync validators], [async validators])`
- While the async validator is running, the control's status is 'PENDING' — use this to show a 'Checking...' spinner in the UI; always add `debounceTime` to avoid firing on every keystroke

Q8: How do you display validation errors in a reactive form?

- Check two conditions before showing an error: the control must be invalid AND it must be touched (user has blurred the field) — this prevents errors showing on an untouched pristine form
- `@if (ctrl.invalid && ctrl.touched) { show error }` — read specific errors from `ctrl.errors?.['required'], ctrl.errors?.['minlength']?.requiredLength`
- Call `form.markAllAsTouched()` when the user clicks Submit — this marks every control as touched and triggers all validation error messages at once
- CSS classes `ng-invalid` and `ng-touched` are automatically added by Angular — you can style them globally without any template conditionals

Q9: What is the purpose of `formGroupName` and `formArrayName` directives?

- `formGroupName` binds a sub-section of the template to a nested `FormGroup` inside a parent `FormGroup` — controls inside the div use their `formControlName` relative to the nested group
- `formArrayName` binds a section of the template to a `FormArray` — inside it, controls are referenced by their numeric index using `[formControlName]='$index'`
- Without these directives, Angular would not know which `FormGroup` a nested `formControlName` belongs to — it would look for the control at the root level and fail to find it
- Both are part of `ReactiveFormsModule` and must be imported in the standalone component's imports array

Q10: How do you dynamically add and remove controls in a `FormArray`?

- Use `push(control)` to add a new `FormControl` or `FormGroup` to the end of the array:
`this.skills.push(this.fb.control("", Validators.required))`
- Use `removeAt(index)` to remove a control by its numeric position:
`this.skills.removeAt(index)`
- Use `insert(index, control)` to add a control at a specific position
- The `FormArray`'s `length` property and controls array always stay in sync with the template — changes are reflected immediately because the template is bound to the same controls array

RxJS & Reactive Programming

Observables, operators, Subjects, and reactive patterns that power Angular's async layer

1. What Is RxJS?

RxJS (Reactive Extensions for JavaScript) is a library for composing asynchronous and event-based programs using observable sequences. Angular is deeply integrated with RxJS — HttpClient, the Router, Reactive Forms, and the async pipe all return or consume Observables.

The core idea is the Observable: a lazy, cancellable stream of values over time. Unlike a Promise which resolves once, an Observable can emit zero, one, or many values and then optionally complete or error. You transform these streams using operators — pure functions that are chained together using `.pipe()`.

Think of an Observable like a conveyor belt. Values appear on the belt over time. Operators are machines along the belt that transform, filter, or combine values. A Subscriber is the worker at the end who does something with each value.

2. Observable, Observer, and Subscription

These three concepts form the foundation of RxJS. Understanding how they relate is essential before working with operators.

```
import { Observable, Subscription } from 'rxjs';

// — Creating an Observable —————
// An Observable is LAZY — no code runs until someone subscribes
const counter$ = new Observable<number>(observer => {
  let count = 0;
  const id = setInterval(() => {
    observer.next(count++);          // emit a value
    if (count > 3) {
      observer.complete();           // signal completion
      clearInterval(id);
    }
  }, 1000);

  // Teardown logic — runs when unsubscribed
  return () => { clearInterval(id); console.log('Cleaned up'); };
});
```

```
// — Subscribing —————
const sub: Subscription = counter$.subscribe({
  next:    (value) => console.log('Value:', value),    // 0, 1, 2, 3
  error:   (err)   => console.error('Error:', err),
  complete: ()      => console.log('Done!'),
});

// — Unsubscribing —————
// Always unsubscribe to prevent memory leaks!
setTimeout(() => sub.unsubscribe(), 2500); // cancels after 2.5s
```

The Dollar Sign Convention

Angular developers suffix Observable variable names with \$ to indicate they are streams rather than plain values. This is a widely adopted convention: `products$` is an `Observable<Product[]>`, `user$` is an `Observable<User>`.

3. Creation Operators — Making Observables

RxJS provides many factory functions to create Observables from different data sources — arrays, events, timers, HTTP calls, and more.

```
import { of, from, interval, timer, fromEvent, EMPTY, NEVER, throwError } from
'rxjs';

// of() — emits each argument as a value, then completes
of(1, 2, 3).subscribe(console.log);           // 1, 2, 3
of([1, 2, 3]).subscribe(console.log);          // [1,2,3] (one array emission)

// from() — converts a Promise, Array, or Iterable into an Observable
from([10, 20, 30]).subscribe(console.log);     // 10, 20, 30
from(fetch('/api/data')).subscribe(r => r.json().then(console.log));
from(Promise.resolve('hello')).subscribe(console.log); // 'hello'

// interval() — emits incrementing numbers every N milliseconds
interval(1000).subscribe(n => console.log(n)); // 0, 1, 2, 3... every second

// timer() — waits then emits (optionally repeats)
timer(2000).subscribe(() => console.log('2s elapsed'));           // once after 2s
timer(0, 1000).subscribe(n => console.log('tick', n));           // immediately,
then every 1s

// fromEvent() — wraps a DOM event as an Observable
const clicks$ = fromEvent<MouseEvent>(document, 'click');
clicks$.subscribe(e => console.log(e.clientX, e.clientY));

// EMPTY — completes immediately, emits nothing
// NEVER — never emits, never completes (use in tests)
// throwError(() => new Error('oops')) — immediately errors
```

4. Pipeable Operators — Transforming Streams

Operators are functions applied inside `.pipe()` that transform the observable stream. They are pure functions — they never modify the source Observable; they return a new one. Angular's `HttpClient` and `Forms` use these operators extensively.

4.1 Transformation Operators

```
import { of, from } from 'rxjs';
import { map, pluck, switchMap, mergeMap, concatMap, exhaustMap } from
  'rxjs/operators';

// map() - transform each emitted value (like Array.map)
of(1, 2, 3).pipe(
  map(n => n * 10)
).subscribe(console.log); // 10, 20, 30

// map with HTTP - extract a nested property
this.http.get<{ data: Product[] }>('/api').pipe(
  map(response => response.data)
).subscribe(products => this.products = products);

// switchMap - CANCEL previous inner observable when new value arrives
// Perfect for: search-as-you-type, route param changes
this.searchTerm$.pipe(
  switchMap(term => this.http.get<Product[]>(`/api/search?q=${term}`))
).subscribe(results => this.results = results);

// concatMap - queue requests, wait for each to COMPLETE before starting next
// Perfect for: sequential operations where order matters
of(1, 2, 3).pipe(
  concatMap(id => this.http.get<Product>(`/api/products/${id}`))
).subscribe(console.log); // product 1, then 2, then 3 - in order

// mergeMap - run ALL inner observables CONCURRENTLY, no ordering
// Perfect for: independent parallel operations
of(1, 2, 3).pipe(
  mergeMap(id => this.http.get<Product>(`/api/products/${id}`))
).subscribe(console.log); // arrives in any order

// exhaustMap - IGNORE new outer values while inner is still running
// Perfect for: submit buttons (ignore double-clicks while request is in flight)
this.submitClicks$.pipe(
  exhaustMap(() => this.http.post('/api/submit', payload))
).subscribe(result => console.log('Submitted once:', result));
```

4.2 Filtering Operators

```
import { filter, take, takeUntil, takeWhile,
```

```
        skip, first, last, distinctUntilChanged } from 'rxjs/operators';
import { Subject } from 'rxjs';

// filter() - only let through values that pass the predicate
of(1, 2, 3, 4, 5).pipe(
  filter(n => n % 2 === 0)
).subscribe(console.log);    // 2, 4

// take(n) - take only the first N values, then complete
interval(500).pipe(take(3)).subscribe(console.log);  // 0, 1, 2

// takeUntil(notifier$) - complete when notifier emits
// ESSENTIAL for preventing memory leaks in components
private destroy$ = new Subject<void>();

ngOnInit() {
  this.someStream$.pipe(
    takeUntil(this.destroy$)
  ).subscribe(val => this.value = val);
}

ngOnDestroy() {
  this.destroy$.next();    // triggers takeUntil to complete
  this.destroy$.complete();
}

// distinctUntilChanged() - skip emission if value is same as previous
// Perfect for: preventing duplicate API calls from form valueChanges
this.searchCtrl.valueChanges.pipe(
  distinctUntilChanged()
).subscribe(term => this.search(term));

// first() - take only the first value and complete
// last() - wait for completion then emit the final value
```

4.3 Utility Operators

```
import { tap, debounceTime, throttleTime, delay,
        catchError, retry, finalize, timeout } from 'rxjs/operators';

// tap() - side effects without modifying the stream (logging, debugging)
this.http.get<Product[]>('/api/products').pipe(
  tap(products => console.log('Fetched:', products.length)),
  tap({ error: err => console.error('Failed:', err) })
).subscribe(this.products);

// debounceTime() - wait N ms after last emission before passing through
// Perfect for: search inputs, autocomplete
this.searchCtrl.valueChanges.pipe(
  debounceTime(400),                // wait 400ms after user stops typing
  distinctUntilChanged(),           // skip if value hasn't changed
  switchMap(term => this.search(term))
).subscribe(results => this.results = results);
```

```
// throttleTime() - emit then silence for N ms (rate limiting)
fromEvent(window, 'scroll').pipe(
  throttleTime(200)
).subscribe(() => this.updateScrollPosition());

// delay() - shift each emission forward in time
of('hello').pipe(delay(1000)).subscribe(console.log); // after 1 second

// timeout() - error if no emission within N ms
this.http.get('/api/slow').pipe(
  timeout(5000),
  catchError(err => of([])) // fallback value on timeout
).subscribe(data => this.data = data);

// finalize() - run teardown logic regardless of complete or error
this.http.get('/api/data').pipe(
  finalize(() => this.isLoading = false)
).subscribe(data => this.data = data);
```

4.4 Combination Operators

```
import { forkJoin, combineLatest, merge, zip,
        withLatestFrom }          from 'rxjs';
import { startWith }              from 'rxjs/operators';

// forkJoin() - wait for ALL to complete, emit combined last values
// Best for: parallel HTTP requests that each fire once
forkJoin({
  user:      this.http.get<User>('/api/user/1'),
  products: this.http.get<Product[]>('/api/products'),
  settings: this.http.get<Settings>('/api/settings'),
}).subscribe(({ user, products, settings }) => {
  this.user      = user;
  this.products = products;
});

// combineLatest() - emit when ANY source emits, using LATEST from all
// Best for: reactive filters - re-fetch when any filter changes
combineLatest([
  this.category$.pipe(startWith('all')),
  this.sortBy$.pipe(startWith('name')),
  this.page$.pipe(startWith(1)),
]).pipe(
  switchMap(([category, sort, page]) =>
    this.productService.getFiltered(category, sort, page))
).subscribe(products => this.products = products);

// merge() - merge multiple streams into one, re-emit as values arrive
merge(clicks$, keydowns$, touchstart$).subscribe(event => {
  this.lastInteraction = Date.now();
});

// withLatestFrom() - combine latest from another stream on each emission
this.saveButton$.pipe(
```

```
withLatestFrom(this.form.valueChanges),
  switchMap(([, formValue]) => this.api.save(formValue))
).subscribe(() => this.showSuccess());
```

5. Subjects — Multicasting and Imperative Emission

A Subject is both an Observable and an Observer. It can emit values imperatively (like an EventEmitter) AND be subscribed to like a regular Observable. Multiple subscribers receive the same emissions — this is called multicasting.

```
import { Subject, BehaviorSubject, ReplaySubject, AsyncSubject } from 'rxjs';

// — Subject — basic multicast —————
// New subscribers MISS any values emitted before they subscribed
const events$ = new Subject<string>();

events$.subscribe(e => console.log('Sub A:', e));
events$.next('click');    // Sub A: click
events$.subscribe(e => console.log('Sub B:', e));
events$.next('scroll');   // Sub A: scroll, Sub B: scroll

// — BehaviorSubject — holds current value —————
// New subscribers IMMEDIATELY get the current (latest) value
// Most common Subject type in Angular — great for state
const user$ = new BehaviorSubject<User | null>(null);

user$.subscribe(u => console.log('User:', u));    // immediately: null
user$.next({ name: 'Gowtham' });                  // User: {name: 'Gowtham'}
user$.getValue();                                  // synchronously: {name:
'Gowtham'}

// — ReplaySubject — replay last N values —————
// New subscribers receive the last N buffered emissions immediately
const log$ = new ReplaySubject<string>(3); // buffer last 3

log$.next('Event 1');
log$.next('Event 2');
log$.next('Event 3');
log$.next('Event 4');
log$.subscribe(e => console.log(e)); // receives: Event 2, 3, 4 (last 3)

// — AsyncSubject — only emits the LAST value on complete —————
const result$ = new AsyncSubject<number>();
result$.next(1);
result$.next(2);
result$.next(3);
result$.subscribe(v => console.log(v)); // nothing yet
result$.complete();                     // now emits: 3 (only last value)
```

6. BehaviorSubject as a State Store

BehaviorSubject is the foundation of lightweight Angular state management. A service exposes the subject's value as an Observable (so consumers cannot emit directly) and provides methods that call `next()` to update the state.

```
// cart.service.ts - simple state store using BehaviorSubject
import { Injectable }      from '@angular/core';
import { BehaviorSubject }  from 'rxjs';
import { map }             from 'rxjs/operators';
import { Product }         from './product.model';

@Injectable({ providedIn: 'root' })
export class CartService {
  // Private BehaviorSubject - only THIS service can call .next()
  private cartItems$ = new BehaviorSubject<Product[]>([]);

  // Public Observable - consumers can only READ the stream
  readonly items$     = this.cartItems$.asObservable();
  readonly itemCount$ = this.items$.pipe(map(items => items.length));
  readonly total$     = this.items$.pipe(
    map(items => items.reduce((sum, p) => sum + p.price, 0))
  );

  addItem(product: Product): void {
    const current = this.cartItems$.getValue();
    this.cartItems$.next([...current, product]); // immutable update
  }

  removeItem(productId: number): void {
    const current = this.cartItems$.getValue();
    this.cartItems$.next(current.filter(p => p.id !== productId));
  }

  clearCart(): void {
    this.cartItems$.next([]);
  }
}

// cart-icon.component.ts - reactive UI with async pipe
export class CartIconComponent {
  private cart = inject(CartService);
  count$ = this.cart.itemCount$;
  total$ = this.cart.total$;
}

// cart-icon.component.html
// <span>{{ count$ | async }} items - {{ total$ | async | currency }}</span>
```

7. Error Handling Operators

Errors in an Observable stream terminate the stream by default. RxJS provides operators to gracefully handle errors — catching them, retrying the source, or substituting a fallback value.

```
import { catchError, retry, retryWhen, throwError, EMPTY, of } from 'rxjs';
import { delay, take } from 'rxjs/operators';

// catchError() - catch and handle the error, return a replacement Observable
this.http.get<Product[]>('/api/products').pipe(
  catchError(err => {
    console.error('Caught:', err.message);
    return of([]); // return fallback empty array - stream continues
    // return EMPTY; // complete without emitting - stream ends silently
    // return throwError(() => err); // re-throw - propagates to subscriber
  })
).subscribe(products => this.products = products);

// retry(n) - automatically resubscribe on error up to N times
this.http.get('/api/flaky').pipe(
  retry(3), // retry 3 times before letting error through
  catchError(err => of([]))
).subscribe(data => this.data = data);

// retry with delay using retryWhen (classic) or retry config object
this.http.get('/api/data').pipe(
  retry({
    count: 3,
    delay: 2000 // wait 2 seconds between retries
  }),
  catchError(err => {
    this.error = 'Failed after 3 retries';
    return EMPTY;
  })
).subscribe(data => this.data = data);
```

8. Memory Leak Prevention — Unsubscribing Strategies

A subscription that is never cleaned up holds a reference to the component even after it is destroyed — causing memory leaks and phantom updates on destroyed views. Angular offers several patterns to prevent this.

```
// — Strategy 1: async pipe - RECOMMENDED —————
// Angular handles subscribe/unsubscribe automatically
@Component({
  template: `@for (p of products$ | async; track p.id) { <div>{{ p.name }}</div>`
})
export class ProductListComponent {
  products$ = inject(ProductService).getAll();
  // No manual subscription, no cleanup needed!
}
```

```
// — Strategy 2: takeUntilDestroyed (Angular 16+) — CLEANEST ———
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

export class DashboardComponent {
  products: Product[] = [];

  constructor() {
    inject(ProductService).getAll().pipe(
      takeUntilDestroyed() // auto-unsubscribes when component is destroyed
    ).subscribe(p => this.products = p);
  }
}

// — Strategy 3: takeUntil with destroy$ Subject (pre-Angular 16) —
export class OldComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();

  ngOnInit() {
    this.someStream$.pipe(
      takeUntil(this.destroy$)
    ).subscribe(val => this.value = val);
  }

  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

// — Strategy 4: Subscription.add() — manage multiple subs ———
export class MultiSubComponent implements OnDestroy {
  private subs = new Subscription();

  ngOnInit() {
    this.subs.add(this.stream1$.subscribe(...));
    this.subs.add(this.stream2$.subscribe(...));
  }

  ngOnDestroy() {
    this.subs.unsubscribe(); // unsubscribes ALL at once
  }
}
```

The async pipe is almost always the best choice. It is declarative, self-cleaning, and triggers change detection correctly. Use `takeUntilDestroyed()` when you must subscribe imperatively and need values in the class (not just the template).

9. Real-World Pattern — Search with Debounce

The search-as-you-type pattern is one of the most common RxJS use cases in Angular. It combines `debounceTime`, `distinctUntilChanged`, and `switchMap` to efficiently query an API as the user types — without flooding the server with every keystroke.

```
// search.component.ts
import { Component, inject } from '@angular/core';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { AsyncPipe } from '@angular/common';
import { debounceTime, distinctUntilChanged,
      switchMap, catchError, startWith } from 'rxjs/operators';
import { of, Observable } from 'rxjs';
import { ProductService, Product } from '../product.service';

@Component({
  standalone: true,
  imports: [ReactiveFormsModule, AsyncPipe],
  template: `
    <input [formControl]='searchCtrl' placeholder='Search products... '>

    @if (results$ | async; as results) {
      @if (results.length === 0) {
        <p>No results found.</p>
      }
      @for (product of results; track product.id) {
        <div>{{ product.name }} - {{ product.price | currency }}</div>
      }
    }
  `
})
export class SearchComponent {
  private productService = inject(ProductService);

  searchCtrl = new FormControl('');

  results$: Observable<Product[]> = this.searchCtrl.valueChanges.pipe(
    startWith(''), // emit immediately on init
    debounceTime(400), // wait 400ms after user stops typing
    distinctUntilChanged(), // skip if value has not changed
    switchMap(term => // cancel previous, start new search
      this.productService.search(term ?? '').pipe(
        catchError(() => of([])) // return empty on error
      )
    )
  );
}
```

Interview Questions & Answers

Q1: What is an Observable and how is it different from a Promise?

- A Promise resolves once with a single value and is eager — it starts executing immediately when created
- An Observable is lazy — nothing runs until you subscribe. It can emit zero, one, or many values over time and can be cancelled by unsubscribing
- Observables are composable — you can chain operators like `map`, `filter`, `debounceTime`, and `switchMap` inside `.pipe()` to transform the stream declaratively
- Observables are cancellable — calling `subscription.unsubscribe()` stops execution and runs teardown logic; Promises cannot be cancelled once started

Q2: What is the difference between `switchMap`, `mergeMap`, `concatMap`, and `exhaustMap`?

- `switchMap` — cancels the previous inner Observable when a new outer value arrives; use for search-as-you-type where only the latest result matters
- `concatMap` — queues inner Observables and processes them one at a time in order; use when sequence matters, like sequential API writes
- `mergeMap` — runs all inner Observables concurrently with no cancellation or queuing; use for independent parallel operations where order does not matter
- `exhaustMap` — ignores new outer emissions while an inner Observable is still active; use for form submit buttons to prevent duplicate submissions

Q3: What is the difference between `Subject`, `BehaviorSubject`, and `ReplaySubject`?

- `Subject` — basic multicast; new subscribers only receive values emitted AFTER they subscribe — they miss previous emissions
- `BehaviorSubject` — holds the current value; new subscribers IMMEDIATELY receive the latest value upon subscribing. Requires an initial value. Most used in Angular services for state
- `ReplaySubject(n)` — buffers the last N emissions; new subscribers immediately receive the last N values upon subscribing
- `AsyncSubject` — only emits the LAST value, and only on `complete()`; rarely used in practice

Q4: What is `debounceTime` and why is it used in search inputs?

- `debounceTime(ms)` delays passing a value through the stream until the source has been silent for the specified duration
- Without it, every keystroke triggers an API call — a user typing 'angular' fires 7 requests when only the last one is needed
- With `debounceTime(400)`, Angular waits until the user pauses typing for 400ms before firing the search — dramatically reducing API calls
- It is almost always paired with `distinctUntilChanged()` to prevent a duplicate request when the user types a character and immediately deletes it, ending with the same value

Q5: What is the async pipe and why is it preferred over manual subscriptions?

- The async pipe subscribes to an Observable in the template and renders the latest emitted value — it automatically unsubscribes when the component is destroyed, preventing memory leaks
- It eliminates boilerplate: no ngOnInit subscription, no ngOnDestroy cleanup, no local property to hold the value
- It also works correctly with OnPush change detection — it marks the component for check whenever the Observable emits a new value
- Prefer: `products$ = this.service.getAll()` in the class and `(products$ | async)` in the template over manually subscribing and storing in `this.products`

Q6: What are the strategies to prevent memory leaks from subscriptions?

- async pipe in the template — Angular handles subscribe and unsubscribe automatically; the cleanest and safest approach
- `takeUntilDestroyed()` (Angular 16+) — pipe operator that automatically completes the stream when the injection context (component) is destroyed; works in constructor and field initializers
- `takeUntil(destroy$)` — classic pattern using a Subject; call `destroy$.next()` and `destroy$.complete()` in `ngOnDestroy`
- `Subscription.add()` — add multiple subscriptions to a parent Subscription and call `parent.unsubscribe()` in `ngOnDestroy` to clean them all up at once

Q7: What is `catchError` and how does it differ from the error callback in `subscribe()`?

- `catchError` is a pipeable operator inside `.pipe()` — it catches the error and returns a replacement Observable, keeping the outer stream alive for subsequent subscribers
- The error callback in `subscribe({ error: fn })` only handles the error at the subscriber level — it cannot return a replacement value, and the stream is terminated
- `catchError` in the service layer means the component's subscription never sees the error — it receives a fallback value (e.g. an empty array) instead
- Best practice: handle errors in the service with `catchError` and `throwError` for re-throwing a cleaned-up message; handle the final error message in the component's error callback

Q8: What is `forkJoin` and when would you use it?

- `forkJoin` waits for all input Observables to complete, then emits a single combined result containing the last value from each
- It is the RxJS equivalent of `Promise.all()` — ideal for firing multiple independent HTTP requests simultaneously and waiting for all of them
- If any source errors before completing, `forkJoin` errors immediately and the results of the other sources are lost — use `catchError` on each inner Observable to handle this
- It will never emit if any source never completes — avoid using `forkJoin` with long-lived streams like `BehaviorSubjects` or `interval()`

Q9: What is the difference between cold and hot Observables?

- Cold Observable — each subscriber gets its own independent execution of the producer; a new HTTP request is made for every subscription. HttpClient returns cold Observables
- Hot Observable — all subscribers share the same underlying producer; values are broadcast to all current subscribers. fromEvent(document, 'click') is hot — the click happens once regardless of how many subscribers exist
- Subject and its variants are hot — they multicast to all current subscribers simultaneously
- shareReplay() converts a cold Observable into a hot one — the underlying HTTP request fires once and is shared across all subscribers

Q10: What is the takeUntilDestroyed operator and how does it work?

- Introduced in Angular 16, takeUntilDestroyed() is a RxJS operator that automatically completes a stream when the enclosing injection context (component, directive, or service) is destroyed
- It must be called within an injection context — typically in the constructor or as a field initializer, not in ngOnInit (unless you pass a DestroyRef manually)
- It replaces the boilerplate takeUntil(destroy\$) + destroy\$.next() + destroy\$.complete() pattern with a single, clean operator call
- You can also use it outside constructor by passing a DestroyRef: takeUntilDestroyed(inject(DestroyRef)) — this works in any function that has access to inject()

Auth (JWT) on the Client Side

Implement login, token storage, route protection, auto-refresh, and logout in Angular

1. How JWT Authentication Works

JSON Web Token (JWT) is the most widely used stateless authentication mechanism in modern SPAs. The flow has three steps: the user logs in with credentials, the server validates them and returns a signed JWT access token (and often a refresh token), and the client stores these tokens and sends the access token with every subsequent API request inside the Authorization header.

A JWT is a Base64-encoded string with three parts separated by dots: Header.Payload.Signature. The payload contains claims — the user ID, roles, email, and expiry time (exp). The server validates requests by verifying the signature using a secret key — no session lookup is needed, making it stateless and scalable.

```
// A decoded JWT payload looks like this:
{
  'sub':    '42',                // subject - user ID
  'email':  'gowtham@app.com',
  'roles':  ['admin', 'user'],
  'iat':    1716000000,          // issued at (Unix timestamp)
  'exp':    1716003600          // expires at (1 hour later)
}

// Authorization header sent with every request:
// Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiI0MiJ9.xxx

// Two-token strategy (industry standard):
// Access Token - short-lived (15 min), sent in every API request
// Refresh Token - long-lived (7-30 days), used ONLY to get new access tokens
```

Never store tokens in `localStorage` if your app is vulnerable to XSS — a malicious script can steal them. The safest option is `HttpOnly` cookies (server-set). If you must use `localStorage`, ensure robust Content Security Policies and sanitize all user input.

2. Token Storage Options

Choosing where to store JWTs involves a trade-off between security and convenience. There are three main options, each with different risk profiles:

```
// — Option 1: localStorage — Simple but XSS-vulnerable —————
// Persists across page refreshes and browser restarts
// Accessible by any JavaScript on the page — XSS risk
localStorage.setItem('accessToken', token);
localStorage.getItem('accessToken');
localStorage.removeItem('accessToken');

// — Option 2: sessionStorage — XSS-vulnerable, tab-scoped —————
// Cleared when the tab is closed — slightly safer than localStorage
// Still accessible by JavaScript
sessionStorage.setItem('accessToken', token);

// — Option 3: Memory (in-service variable) — Safest for XSS —————
// Token lives in a JavaScript variable — no storage API
// Lost on page refresh — typically paired with a refresh token in
// an HttpOnly cookie that the server can use to silently re-issue
private accessToken: string | null = null;

// — Option 4: HttpOnly Cookie (Server-Set) — Best Security —————
// Set by the server with Set-Cookie: token=...; HttpOnly; Secure; SameSite=Strict
// NOT accessible from JavaScript at all — immune to XSS
// The browser automatically sends it with every same-origin request
// Angular must send { withCredentials: true } on HTTP requests
this.http.get('/api/data', { withCredentials: true })
```

3. AuthService — The Core of Client-Side Auth

The AuthService is the single source of truth for authentication state. It handles login, logout, token storage, token decoding, and expiry checks. All other parts of the app — guards, interceptors, and components — consume this service.

```
// auth.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { BehaviorSubject, tap, Observable } from 'rxjs';

export interface AuthTokens {
  accessToken: string;
  refreshToken: string;
}

export interface DecodedToken {
  sub: string;
  email: string;
  roles: string[];
  exp: number;
  iat: number;
}

export interface User {
  id: string;
```

```
    email: string;
    roles: string[];
  }

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);

  private readonly ACCESS_TOKEN_KEY = 'access_token';
  private readonly REFRESH_TOKEN_KEY = 'refresh_token';

  // BehaviorSubject - current user is immediately available to any subscriber
  private currentUser$ = new BehaviorSubject<User | null>(this.loadUserFromToken());

  // Public read-only stream
  readonly user$ = this.currentUser$.asObservable();
  readonly isLoggedIn$ = this.user$.pipe(map(u => u !== null));

  // --- Login -----
  login(email: string, password: string): Observable<AuthTokens> {
    return this.http.post<AuthTokens>('/api/auth/login', { email, password })
      .pipe(
        tap(tokens => this.storeTokens(tokens))
      );
  }

  // --- Logout -----
  logout(): void {
    localStorage.removeItem(this.ACCESS_TOKEN_KEY);
    localStorage.removeItem(this.REFRESH_TOKEN_KEY);
    this.currentUser$.next(null);
    this.router.navigate(['/login']);
  }

  // --- Token Accessors -----
  getAccessToken(): string | null {
    return localStorage.getItem(this.ACCESS_TOKEN_KEY);
  }

  getRefreshToken(): string | null {
    return localStorage.getItem(this.REFRESH_TOKEN_KEY);
  }

  isLoggedIn(): boolean {
    const token = this.getAccessToken();
    return token !== null && !this.isTokenExpired(token);
  }

  // --- Private Helpers -----
  private storeTokens(tokens: AuthTokens): void {
    localStorage.setItem(this.ACCESS_TOKEN_KEY, tokens.accessToken);
    localStorage.setItem(this.REFRESH_TOKEN_KEY, tokens.refreshToken);
    const decoded = this.decodeToken(tokens.accessToken);
    if (decoded) {
```

```
        this.currentUser$.next({
          id: decoded.sub,
          email: decoded.email,
          roles: decoded.roles,
        });
      }
    }

    // Decode JWT payload WITHOUT a library (pure client-side)
    decodeToken(token: string): DecodedToken | null {
      try {
        const payload = token.split('.')[1];
        const decoded = atob(payload.replace(/-/g, '+').replace(/_/g, '/'));
        return JSON.parse(decoded) as DecodedToken;
      } catch {
        return null;
      }
    }

    isTokenExpired(token: string): boolean {
      const decoded = this.decodeToken(token);
      if (!decoded) return true;
      // exp is in seconds; Date.now() is in ms
      return decoded.exp * 1000 < Date.now();
    }

    private loadUserFromToken(): User | null {
      const token = localStorage.getItem(this.ACCESS_TOKEN_KEY);
      if (!token || this.isTokenExpired(token)) return null;
      const decoded = this.decodeToken(token);
      if (!decoded) return null;
      return { id: decoded.sub, email: decoded.email, roles: decoded.roles };
    }
  }
}
```

4. Login Component

The login form collects credentials, calls `AuthService.login()`, and redirects to the intended page. If authentication fails, an error message is shown. The `returnUrl` query param is used to redirect the user back to the page they were trying to visit before being redirected to login.

```
// login.component.ts
import { Component, inject }           from '@angular/core';
import { ReactiveFormsModule,          from '@angular/forms';
      FormBuilder, Validators }
import { Router, ActivatedRoute }      from '@angular/router';
import { AuthService }                 from '../auth.service';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [ReactiveFormsModule],
```

```

template: `
  <form [formGroup]='form' (ngSubmit)='onSubmit()'>
    <h2>Sign In</h2>

    @if (error) {
      <div class='alert alert-error'>{{ error }}</div>
    }

    <input formControlName='email' type='email' placeholder='Email'>
    <input formControlName='password' type='password' placeholder='Password'>

    <button type='submit' [disabled]='form.invalid || isLoading'>
      {{ isLoading ? 'Signing in...' : 'Sign In' }}
    </button>
  </form>
`
))
export class LoginComponent {
  private fb = inject(FormBuilder);
  private auth = inject(AuthService);
  private router = inject(Router);
  private route = inject(ActivatedRoute);

  isLoading = false;
  error = '';

  form = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]],
  });

  onSubmit(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }

    this.isLoading = true;
    this.error = '';

    const { email, password } = this.form.value;

    this.auth.login(email!, password!).subscribe({
      next: () => {
        // Redirect to the page the user was trying to access, or dashboard
        const returnUrl = this.route.snapshot.queryParamMap.get('returnUrl') ??
        '/dashboard';
        this.router.navigateByUrl(returnUrl);
      },
      error: (err) => {
        this.error = err.status === 401 ? 'Invalid email or password' : 'Login
failed. Try again.';
        this.isLoading = false;
      }
    });
  }
}

```

5. Auth Interceptor — Attach Token to Every Request

The auth interceptor automatically attaches the JWT access token to every outgoing HTTP request. This means no individual service needs to manually add headers — the interceptor handles it globally for the entire application.

```
// auth.interceptor.ts
import { HttpInterceptorFn, HttpResponse } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError } from 'rxjs/operators';
import { throwError } from 'rxjs';
import { AuthService } from '../auth.service';
import { Router } from '@angular/router';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const token = auth.getAccessToken();

  // Skip adding token for auth endpoints themselves
  const isAuthEndpoint = req.url.includes('/auth/login')
    || req.url.includes('/auth/refresh');

  // Attach Bearer token if available and not an auth endpoint
  const authReq = token && !isAuthEndpoint
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    catchError((err: HttpResponse) => {
      if (err.status === 401 && !isAuthEndpoint) {
        // Token is invalid or expired — log the user out
        auth.logout();
      }
      return throwError(() => err);
    })
  );
};

// main.ts — register the interceptor
bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
});
```

6. Automatic Token Refresh

Access tokens are short-lived (typically 15 minutes). When one expires, the app should silently obtain a new one using the refresh token — without logging the user out. This is done inside the interceptor: catch a 401 error, call the refresh endpoint, retry the original request with the new token.

```
// token-refresh.interceptor.ts
import { HttpInterceptorFn, HttpErrorResponse,
        HttpRequest, HttpHandlerFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { BehaviorSubject, throwError } from 'rxjs';
import { catchError, filter, switchMap, take } from 'rxjs/operators';
import { AuthService } from '../auth.service';

let isRefreshing = false;
const refreshDone$ = new BehaviorSubject<string | null>(null);

export const tokenRefreshInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);

  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      const isAuthUrl = req.url.includes('/auth/');

      if (err.status !== 401 || isAuthUrl) {
        return throwError(() => err); // not a token error - pass through
      }

      if (isRefreshing) {
        // Another request is already refreshing - queue this one
        return refreshDone$.pipe(
          filter(token => token !== null),
          take(1),
          switchMap(newToken =>
            next(addToken(req, newToken!))
          )
        );
      }

      isRefreshing = true;
      refreshDone$.next(null);

      const refreshToken = auth.getRefreshToken();
      if (!refreshToken) {
        auth.logout();
        return throwError(() => err);
      }

      // Call refresh endpoint
      return auth.refreshTokens(refreshToken).pipe(
        switchMap(tokens => {
          isRefreshing = false;
          refreshDone$.next(tokens.accessToken);
          return next(addToken(req, tokens.accessToken)); // retry original
        }),
        catchError(refreshErr => {
          isRefreshing = false;
          auth.logout(); // refresh failed - force re-login
          return throwError(() => refreshErr);
        })
      );
    })
  );
};
```

```
    })
  );
};

// Helper - clone request with new token
function addToken(req: HttpRequest<unknown>, token: string) {
  return req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
}

// auth.service.ts - add refresh method
refreshTokens(refreshToken: string): Observable<AuthTokens> {
  return this.http.post<AuthTokens>('/api/auth/refresh',
    { refreshToken }
  ).pipe(tap(tokens => this.storeTokens(tokens)));
}
```

The `isRefreshing` flag and `BehaviorSubject` queue are critical. Without them, multiple concurrent 401 requests would each trigger a separate refresh call — causing a race condition where all but one would fail because the refresh token can only be used once.

7. Route Guards — Protecting Pages

Route guards use `AuthService` to decide whether a navigation should proceed. The auth guard protects authenticated-only routes; the role guard restricts access based on user roles; the no-auth guard redirects logged-in users away from the login page.

```
// auth.guard.ts - block unauthenticated users
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.isLoggedIn()) return true;

  return router.createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url } // remember intended destination
  });
};

// role.guard.ts - restrict by role
export const roleGuard = (requiredRole: string): CanActivateFn =>
() => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const user = auth.getCurrentUser();
```

```
    if (user?.roles.includes(requiredRole)) return true;

    return router.createUrlTree(['/forbidden']);
  };

// no-auth.guard.ts - redirect already-logged-in users away from /login
export const noAuthGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (!auth.isLoggedIn()) return true;

  return router.createUrlTree(['/dashboard']);
};

// app.routes.ts - apply guards
export const APP_ROUTES: Routes = [
  { path: 'login',      component: LoginComponent,
    canActivate: [noAuthGuard] },
  { path: 'dashboard', component: DashboardComponent,
    canActivate: [authGuard] },
  { path: 'admin',      component: AdminComponent,
    canActivate: [authGuard, roleGuard('admin')] },
];
```

8. Displaying Auth State in the UI

Components subscribe to the AuthService's `user$` Observable to show/hide elements based on login state. The async pipe keeps the UI reactive — it automatically updates whenever the user logs in or out.

```
// navbar.component.ts
import { Component, inject } from '@angular/core';
import { AsyncPipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { AuthService } from '../auth.service';

@Component({
  selector: 'app-navbar',
  standalone: true,
  imports: [AsyncPipe, RouterLink],
  template: `
    <nav>
      <a routerLink="/">Home</a>

      @if (auth.user$ | async; as user) {
        <!-- Logged in -->
        <span>Hello, {{ user.email }}</span>

        @if (user.roles.includes('admin')) {
          <a routerLink="/admin">Admin Panel</a>
        }
      }
    </nav>
  `
})
```

```
    }

    <button (click)='logout()'>Log Out</button>
  } @else {
    <!-- Not logged in -->
    <a routerLink='/login'>Log In</a>
    <a routerLink='/register'>Sign Up</a>
  }
</nav>
,
}))
export class NavbarComponent {
  auth = inject(AuthService);

  logout(): void {
    this.auth.logout();
  }
}
```

9. Persisting Auth State Across Page Refreshes

When the user refreshes the page, the Angular app re-bootstraps from scratch and all in-memory state is lost. To restore the authenticated session, the AuthService must read the stored token from localStorage on startup and reconstruct the user object from it.

```
// auth.service.ts - initialization on app startup
@Injectable({ providedIn: 'root' })
export class AuthService {
  // Called immediately at service creation time
  // BehaviorSubject initialises by reading the stored token
  private currentUser$ = new BehaviorSubject<User | null>(
    this.loadUserFromToken() // reads from localStorage on startup
  );

  private loadUserFromToken(): User | null {
    const token = localStorage.getItem('access_token');

    // No token stored
    if (!token) return null;

    // Token exists but is already expired
    if (this.isTokenExpired(token)) {
      localStorage.removeItem('access_token');
      localStorage.removeItem('refresh_token');
      return null;
    }

    // Token is valid - reconstruct user from payload
    const decoded = this.decodeToken(token);
    if (!decoded) return null;
  }
}
```

```
    return {
      id: decoded.sub,
      email: decoded.email,
      roles: decoded.roles,
    };
  }

  // Optional: check and proactively refresh on app init
  // Called in app.component.ts constructor
  checkAndRefreshOnStartup(): void {
    const access = this.getAccessToken();
    const refresh = this.getRefreshToken();

    if (!access && refresh) {
      // Access token missing/expired but refresh token exists - silent refresh
      this.refreshTokens(refresh).subscribe({
        error: () => this.logout() // refresh also expired - force login
      });
    }
  }
}

// app.component.ts - trigger startup check
@Component({ selector: 'app-root', standalone: true, template: '<router-outlet/>' })
export class AppComponent {
  constructor() {
    inject(AuthService).checkAndRefreshOnStartup();
  }
}
```

10. Full Auth Flow — Putting It All Together

Here is a summary of how every piece connects in a complete Angular JWT auth implementation:

```
// — Complete Auth Architecture Overview —————

// 1. AuthService (auth.service.ts)
//   - login()           → POST /api/auth/login → stores tokens, emits user
//   - logout()          → clears storage, emits null, navigates to /login
//   - refreshTokens() → POST /api/auth/refresh → stores new tokens
//   - isLoggedIn()     → checks token exists and is not expired
//   - user$             → BehaviorSubject<User | null> for reactive UI

// 2. authInterceptor (auth.interceptor.ts)
//   - Clones every request and adds Authorization: Bearer <token>
//   - On 401: triggers tokenRefreshInterceptor or calls auth.logout()

// 3. tokenRefreshInterceptor (token-refresh.interceptor.ts)
//   - On 401: calls refresh endpoint, retries original request
//   - Queues concurrent 401s to prevent duplicate refresh calls
```

```
// 4. Route Guards
//   - authGuard      → blocks unauthenticated access, saves returnUrl
//   - roleGuard      → restricts by user role
//   - noAuthGuard     → redirects logged-in users away from /login

// 5. LoginComponent (login.component.ts)
//   - Collects credentials via ReactiveForm
//   - Calls auth.login(), on success navigates to returnUrl
//   - Shows server error messages

// 6. NavbarComponent (navbar.component.ts)
//   - Subscribes to auth.user$ via async pipe
//   - Shows user info, role-based links, and logout button

// main.ts — register everything
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(APP_ROUTES),
    provideHttpClient(
      withInterceptors([authInterceptor, tokenRefreshInterceptor])
    )
  ]
});
```

Interview Questions & Answers

Q1: How does JWT authentication work in an Angular SPA?

- The user submits credentials; the server validates them and returns an access token and a refresh token
- The Angular app stores these tokens (in localStorage, sessionStorage, or memory) and attaches the access token to every subsequent API request in the Authorization: Bearer header
- The server validates the token's signature on every request — no session lookup needed, making it stateless and scalable
- When the access token expires, the app uses the refresh token to silently obtain a new access token without forcing the user to log in again

Q2: What are the security risks of storing JWTs in localStorage?

- XSS (Cross-Site Scripting) — if any script on the page can execute malicious code, it can read localStorage and steal the token: `localStorage.getItem('access_token')`
- localStorage tokens persist until explicitly removed — even after the browser is closed and reopened, giving attackers a larger window of opportunity
- HttpOnly cookies are immune to JavaScript access entirely — if the server sets the token as an HttpOnly cookie, no script can read it

- If localStorage must be used, mitigate XSS risk with a strict Content-Security-Policy header, Angular's built-in template sanitisation, and avoiding innerHTML with user content

Q3: What is the purpose of the two-token strategy — access token and refresh token?

- Access tokens are short-lived (5–15 minutes) to limit the damage window if stolen — even if intercepted, they expire quickly
- Refresh tokens are long-lived (days to weeks) and stored more securely; they are only sent to the /auth/refresh endpoint, not to every API endpoint
- When the access token expires, the client silently exchanges the refresh token for a new access token — the user never notices the session being renewed
- If the refresh token is also expired or revoked, the user is forced to log in again — providing a natural, secure session boundary

Q4: How do you decode a JWT on the client side without a library?

- A JWT has three Base64URL-encoded parts separated by dots: header.payload.signature
- Split on '.', take index [1] (the payload), replace URL-safe characters (- to +, _ to /), then call atob() to Base64-decode it, then JSON.parse() the result
- Code: `const payload = JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')))`
- Never trust the decoded payload for security decisions on the server — anyone can decode a JWT; only the server can verify the signature. Client-side decoding is only for reading user info like email and roles for display purposes

Q5: What is an HTTP interceptor's role in JWT authentication?

- The auth interceptor automatically clones every outgoing request and adds the Authorization: Bearer token header — no individual service needs to manually attach headers
- It also listens for 401 Unauthorized responses and either triggers a token refresh or calls auth.logout() to force re-authentication
- Interceptors are registered globally with `provideHttpClient(withInterceptors([authInterceptor]))` and run transparently for all HTTP calls in the app
- The request must be cloned (`req.clone()`) because `HttpRequest` objects are immutable — you cannot modify the original

Q6: How do you handle token refresh when multiple concurrent requests fail with 401?

- Without synchronisation, each of 5 concurrent 401 responses would trigger its own refresh call — but the refresh token can only be used once, so 4 of the 5 would fail

- The solution is an `isRefreshing` flag and a `BehaviorSubject` queue: the first 401 triggers the refresh; all other 401s wait on the `BehaviorSubject` until the refresh completes
- Once refresh succeeds, the new access token is emitted to the `BehaviorSubject`, which queued requests use to retry with the new token
- If refresh fails, call `auth.logout()` to clear all tokens and redirect to the login page

Q7: What is the difference between the `authGuard` and the `noAuthGuard`?

- `authGuard` protects routes that require authentication — if the user is NOT logged in, it redirects to `/login` and saves the intended URL as a `returnUrl` query param
- `noAuthGuard` protects routes that should only be accessible when NOT logged in (e.g., `/login`, `/register`) — if the user IS already logged in, it redirects to `/dashboard`
- Without `noAuthGuard`, a logged-in user could manually navigate to `/login`, which creates a confusing UX and potentially clears their session if they submit the form
- Both use the same `AuthService.isLoggedIn()` check but redirect in opposite directions

Q8: How do you restore the authenticated session after a page refresh?

- On page refresh, Angular re-bootstraps and all in-memory state is lost — the `BehaviorSubject` resets to its initial value
- The `AuthService` initialises its `BehaviorSubject` by calling a `loadUserFromToken()` method that reads from `localStorage`, checks token expiry, and decodes the payload to reconstruct the `User` object
- If the access token is expired but a refresh token exists, the app can silently call the refresh endpoint on startup to obtain a new access token before any protected route is activated
- This startup check is best triggered in the `AppComponent` constructor so it runs once, before any route guard activates

Q9: How do you implement role-based access control (RBAC) in Angular?

- Store user roles in the JWT payload (e.g. `roles: ['admin', 'editor']`) — the `AuthService` decodes them on login and makes them available via the `user$ Observable`
- Create a `roleGuard` factory function that accepts a required role and returns a `CanActivateFn` that checks `user.roles.includes(requiredRole)`
- Apply it in routes: `{ path: 'admin', canActivate: [authGuard, roleGuard('admin')] }` — both guards must pass for the route to activate
- In templates, use `@if (user?.roles.includes('admin'))` to conditionally show admin-only UI elements — but always enforce RBAC on the server too, since client-side checks can be bypassed

Q10: What happens to the JWT authentication flow when the access token expires mid-session?

- The next API request returns 401 Unauthorized — the server rejects the expired token

- The `tokenRefreshInterceptor` catches the 401, calls the `/auth/refresh` endpoint with the refresh token, and receives a new access token
- The interceptor then retries the original failed request with the new token — the component that triggered the request never knows the token was refreshed, the response arrives transparently
- If the refresh token is also expired or invalid (e.g., the user has been inactive for 30 days), the refresh call fails, `auth.logout()` is called, and the user is redirected to `/login`

State Management

BehaviorSubject services, Angular Signals, NgRx, and choosing the right tool for your app

1. What Is State and Why Does It Need Managing?

State is any data your application needs to remember and display — the logged-in user, the items in a shopping cart, the result of the last API call, which tab is selected, or whether a modal is open. As apps grow, state becomes spread across many components and keeping it synchronised becomes a major source of bugs.

State management is the discipline of deciding where state lives, how it changes, and how components are notified of changes. Angular provides a spectrum of tools — from simple shared services with BehaviorSubject to the lightweight Signals API introduced in Angular 16 to full Redux-style libraries like NgRx — and choosing the right level of complexity for your app is the key skill.

Golden rule: use the simplest solution that solves the problem. A single BehaviorSubject in a service handles most use cases. Reach for NgRx only when your state interactions are truly complex — multiple slices of state that interact, time-travel debugging needs, or large teams that need strict conventions.

```
// Types of state in a typical Angular app:

// 1. Server state    - data fetched from an API (products, users, orders)
//    Tool: service + BehaviorSubject, or NgRx, or signals

// 2. UI state        - which tab is open, modal visibility, loading spinners
//    Tool: component-local state (class property or signal)

// 3. Session state   - auth token, user preferences, language setting
//    Tool: singleton service + BehaviorSubject + localStorage

// 4. Form state       - draft values, validation errors
//    Tool: Angular Reactive Forms (FormGroup)

// 5. Router state     - current URL, query params
//    Tool: Angular Router (ActivatedRoute)
```

2. Approach 1 — Service with BehaviorSubject

The most common Angular state management pattern needs no extra libraries. A singleton service holds a BehaviorSubject for each piece of state, exposes it as a read-only Observable, and provides methods that call `.next()` to update it. Any component that injects the service gets a live, reactive view of that state.

2.1 Building a Typed State Service

```
// product-state.service.ts
import { Injectable }      from '@angular/core';
import { BehaviorSubject } from 'rxjs';
import { map }             from 'rxjs/operators';
import { Product }         from './product.model';

export interface ProductState {
  items:      Product[];
  isLoading: boolean;
  error:      string | null;
  selected:   Product | null;
}

const INITIAL_STATE: ProductState = {
  items:      [],
  isLoading: false,
  error:      null,
  selected:   null,
};

@Injectable({ providedIn: 'root' })
export class ProductStateService {
  // Private — only this service can mutate state
  private state$ = new BehaviorSubject<ProductState>(INITIAL_STATE);

  // Public derived streams — components subscribe to only what they need
  readonly items$      = this.state$.pipe(map(s => s.items));
  readonly isLoading$  = this.state$.pipe(map(s => s.isLoading));
  readonly error$       = this.state$.pipe(map(s => s.error));
  readonly selected$    = this.state$.pipe(map(s => s.selected));
  readonly count$       = this.state$.pipe(map(s => s.items.length));

  // Immutable update helper — spreads existing state, overrides only what changed
  private setState(partial: Partial<ProductState>): void {
    this.state$.next({ ...this.state$.getValue(), ...partial });
  }

  // — Actions (methods that describe what happened) —————
  setLoading(isLoading: boolean) { this.setState({ isLoading }); }
  setError(error: string | null) { this.setState({ error }); }
  setProducts(items: Product[]) { this.setState({ items, isLoading: false }); }
  selectProduct(p: Product | null) { this.setState({ selected: p }); }

  addProduct(product: Product): void {
    const items = [...this.state$.getValue().items, product];
    this.setState({ items });
  }
}
```

```
updateProduct(updated: Product): void {
  const items = this.state$.getValue().items
    .map(p => p.id === updated.id ? updated : p);
  this.setState({ items });
}

removeProduct(id: number): void {
  const items = this.state$.getValue().items.filter(p => p.id !== id);
  this.setState({ items });
}

reset(): void { this.state$.next(INITIAL_STATE); }
}
```

2.2 Connecting the Service to a Component

```
// product-list.component.ts
import { Component, inject, OnInit } from '@angular/core';
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import { ProductStateService } from '../product-state.service';
import { ProductApiService } from '../product-api.service';

@Component({
  selector: 'app-product-list',
  standalone: true,
  imports: [AsyncPipe, CurrencyPipe],
  template: `
    @if (state.isLoading$ | async) {
      <p>Loading...</p>
    }
    @if (state.error$ | async; as error) {
      <p class='error'>{{ error }}</p>
    }
    @for (product of state.items$ | async; track product.id) {
      <div class='card' (click)='select(product)'>
        <h3>{{ product.name }}</h3>
        <span>{{ product.price | currency }}</span>
        <button (click)='remove(product.id, $event)'>Remove</button>
      </div>
    }
    <p>Total: {{ state.count$ | async }} products</p>
  `
})
export class ProductListComponent implements OnInit {
  state = inject(ProductStateService);
  private api = inject(ProductApiService);

  ngOnInit() {
    this.state.setLoading(true);
    this.api.getAll().subscribe({
      next: products => this.state.setProducts(products),
      error: err => this.state.setError(err.message),
    });
  }
}
```

```

    select(product: Product)      { this.state.selectProduct(product); }
    remove(id: number, e: Event)  { e.stopPropagation();
this.state.removeProduct(id); }
}

```

3. Approach 2 — Angular Signals (Angular 16+)

Signals are Angular's built-in fine-grained reactivity primitive introduced in Angular 16. A signal holds a value and automatically notifies any consumer (template, computed, or effect) when that value changes — without requiring RxJS, subscriptions, or manual change detection.

Signals integrate directly with Angular's change detection. Unlike Observables, they are synchronous and always hold a current value (like BehaviorSubject). They are the modern, Angular-native way to manage local and shared component state.

3.1 signal(), computed(), and effect()

```

import { Component, signal, computed, effect } from '@angular/core';

@Component({
  standalone: true,
  template: `
    <p>Count: {{ count() }}</p>           <!-- call the signal like a function -->
    <p>Double: {{ double() }}</p>         <!-- computed updates automatically -->
    <p>Status: {{ status() }}</p>
    <button (click)='increment()'>+1</button>
    <button (click)='decrement()'>-1</button>
    <button (click)='reset()'>Reset</button>
  `,
})
export class CounterComponent {
  // signal(initialValue) - writable signal
  count = signal(0);
  name = signal('Angular');

  // computed(() => ...) - derived, read-only; recalculates when dependencies
  // change
  double = computed(() => this.count() * 2);
  status = computed(() => this.count() > 0 ? 'positive' : 'zero or negative');

  // effect(() => ...) - side effect that runs when any signal it reads changes
  constructor() {
    effect(() => {
      console.log(`Count changed to: ${this.count()}`);
      // Auto-tracks count signal - re-runs whenever count() changes
    });
  }

  increment() { this.count.update(n => n + 1); } // update via function
  decrement() { this.count.update(n => n - 1); }
  reset()      { this.count.set(0); }           // set directly
}

```

3.2 Signal-Based State Service

```
// cart.service.ts - signal-based state
import { Injectable, signal, computed } from '@angular/core';
import { Product } from '../product.model';

@Injectable({ providedIn: 'root' })
export class CartService {
  // Private writable signal - only this service mutates it
  private items = signal<Product[]>([]);

  // Public read-only derived signals
  readonly cartItems = this.items.asReadonly();
  readonly itemCount = computed(() => this.items().length);
  readonly totalPrice = computed(() =>
    this.items().reduce((sum, p) => sum + p.price, 0)
  );
  readonly isEmpty = computed(() => this.items().length === 0);

  addItem(product: Product): void {
    this.items.update(current => [...current, product]);
  }

  removeItem(id: number): void {
    this.items.update(current => current.filter(p => p.id !== id));
  }

  clearCart(): void {
    this.items.set([]);
  }
}

// cart-icon.component.ts - no async pipe needed with signals!
@Component({
  standalone: true,
  template: `
    <span>{{ cart.itemCount() }} items</span>
    <span>{{ cart.totalPrice() | currency }}</span>
  `
})
export class CartIconComponent {
  cart = inject(CartService);
}
```

3.3 Converting Between Signals and Observables

```
import { toSignal, toObservable } from '@angular/core/rxjs-interop';

// toSignal() - convert an Observable into a Signal
// Angular auto-unsubscribes when the component is destroyed
@Component({ standalone: true, template: `{{ products() | json }}` })
export class ProductComponent {
```

```
private productService = inject(ProductService);

// Observable → Signal (must be called in injection context)
products = toSignal(
  this.productService.getAll(),
  { initialValue: [] as Product[] } // initial value before Observable emits
);

// toObservable() — convert a Signal into an Observable
@Injectable({ providedIn: 'root' })
export class SearchService {
  searchTerm = signal('');

  // Signal → Observable (for debouncing, switchMap, etc.)
  results$ = toObservable(this.searchTerm).pipe(
    debounceTime(400),
    distinctUntilChanged(),
    switchMap(term => this.http.get<Product[]>(`/api/search?q=${term}`))
  );
}
```

4. Approach 3 — NgRx (Redux Pattern)

NgRx is the most popular full-featured state management library for Angular. It implements the Redux pattern: state is immutable and stored in a single Store, changes are described by Actions, and pure Reducer functions produce the new state. Effects handle side effects like HTTP calls.

NgRx shines in large, complex applications with many developers — it enforces strict conventions, provides excellent DevTools for time-travel debugging, and makes state transitions completely explicit and traceable.

4.1 Core Concepts — Actions, Reducers, Selectors, Effects

```
// NgRx installation
// npm install @ngrx/store @ngrx/effects @ngrx/store-devtools

// — 1. ACTIONS — describe what happened —————
// product.actions.ts
import { createAction, props } from '@ngrx/store';
import { Product } from '../product.model';

export const loadProducts = createAction('[Product] Load Products');
export const loadProductsSuccess = createAction(
  '[Product] Load Products Success',
  props<{ products: Product[] }>()
);
export const loadProductsFailure = createAction(
  '[Product] Load Products Failure',
  props<{ error: string }>()
);
```

```
);
export const addProduct      = createAction('[Product] Add',      props<{ product:
Product }>());
export const removeProduct = createAction('[Product] Remove', props<{ id: number
}>());
export const selectProduct = createAction('[Product] Select', props<{ id: number
}>());
```

```
// — 2. REDUCER — pure function: (state, action) => newState ———
// product.reducer.ts
import { createReducer, on } from '@ngrx/store';
import * as ProductActions from '../product.actions';

export interface ProductState {
  products:    Product[];
  selectedId:  number | null;
  isLoading:   boolean;
  error:       string | null;
}

const initialState: ProductState = {
  products:    [],
  selectedId:  null,
  isLoading:   false,
  error:       null,
};

export const productReducer = createReducer(
  initialState,

  on(ProductActions.loadProducts, state => ({
    ...state, isLoading: true, error: null
  })),

  on(ProductActions.loadProductsSuccess, (state, { products }) => ({
    ...state, products, isLoading: false
  })),

  on(ProductActions.loadProductsFailure, (state, { error }) => ({
    ...state, error, isLoading: false
  })),

  on(ProductActions.addProduct, (state, { product }) => ({
    ...state, products: [...state.products, product]
  })),

  on(ProductActions.removeProduct, (state, { id }) => ({
    ...state, products: state.products.filter(p => p.id !== id)
  })),

  on(ProductActions.selectProduct, (state, { id }) => ({
    ...state, selectedId: id
  })),
);
```

```
// — 3. SELECTORS — memoised queries against the store —————
// product.selectors.ts
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { ProductState } from './product.reducer';

// Feature selector — root slice of state
const selectProductFeature = createFeatureSelector<ProductState>('products');

// Derived selectors — memoised, only recalculate when input changes
export const selectAllProducts = createSelector(
  selectProductFeature, s => s.products
);
export const selectIsLoading = createSelector(
  selectProductFeature, s => s.isLoading
);
export const selectError = createSelector(
  selectProductFeature, s => s.error
);
export const selectSelectedProduct = createSelector(
  selectProductFeature, s =>
    s.products.find(p => p.id === s.selectedId) ?? null
);
export const selectProductCount = createSelector(
  selectAllProducts, products => products.length
);
```

```
// — 4. EFFECTS — handle side effects (HTTP calls) —————
// product.effects.ts
import { Injectable, inject } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { switchMap, map, catchError } from 'rxjs/operators';
import { of } from 'rxjs';
import * as ProductActions from './product.actions';
import { ProductApiService } from './product-api.service';

@Injectable()
export class ProductEffects {
  private actions$ = inject(Actions);
  private api = inject(ProductApiService);

  // Listens for loadProducts action → calls API → dispatches success or failure
  loadProducts$ = createEffect(() =>
    this.actions$.pipe(
      ofType(ProductActions.loadProducts),
      switchMap(() =>
        this.api.getAll().pipe(
          map(products => ProductActions.loadProductsSuccess({ products })),
          catchError(error => of(ProductActions.loadProductsFailure(
            { error: error.message }
          )))
        )
      )
    )
  );
};
```

```
}
```

4.2 Using the Store in a Component

```
// product-list.component.ts - NgRx component
import { Component, inject, OnInit } from '@angular/core';
import { Store } from '@ngrx/store';
import { AsyncPipe, CurrencyPipe } from '@angular/common';
import * as ProductActions from '../product.actions';
import * as ProductSelectors from '../product.selectors';

@Component({
  standalone: true,
  imports: [AsyncPipe, CurrencyPipe],
  template: `
    @if (isLoading$ | async) { <p>Loading...</p> }
    @if (error$ | async; as err) { <p class='error'>{{ err }}</p> }

    @for (p of products$ | async; track p.id) {
      <div (click)='select(p.id)'>
        {{ p.name }} - {{ p.price | currency }}
        <button (click)='remove(p.id, $event)'>x</button>
      </div>
    }
  `,
})
export class ProductListComponent implements OnInit {
  private store = inject(Store);

  // Select slices of state as Observables
  products$ = this.store.select(ProductSelectors.selectAllProducts);
  isLoading$ = this.store.select(ProductSelectors.selectIsLoading);
  error$ = this.store.select(ProductSelectors.selectError);

  ngOnInit() {
    // Dispatch an action - the effect handles the API call
    this.store.dispatch(ProductActions.loadProducts());
  }

  select(id: number) {
    this.store.dispatch(ProductActions.selectProduct({ id }));
  }

  remove(id: number, e: Event) {
    e.stopPropagation();
    this.store.dispatch(ProductActions.removeProduct({ id }));
  }
}

// main.ts - register NgRx
bootstrapApplication(AppComponent, {
  providers: [
    provideStore({ products: productReducer }),
    provideEffects([ProductEffects]),
  ],
});
```

```
    provideStoreDevtools({ maxAge: 25 }), // browser DevTools extension
  ]
});
```

5. Approach 4 — NgRx Signal Store (Modern NgRx)

NgRx Signal Store (part of `@ngrx/signals`, released with NgRx 17) is a lightweight alternative to the classic Store/Actions/Reducers pattern. It uses Angular Signals under the hood, requires far less boilerplate, and is the recommended approach for new Angular 17+ projects that want NgRx patterns without the ceremony.

```
// npm install @ngrx/signals

// product.store.ts - NgRx Signal Store
import { signalStore, withState, withMethods,
  withComputed, patchState } from '@ngrx/signals';
import { inject, computed } from '@angular/core';
import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { switchMap, tap } from 'rxjs/operators';
import { pipe } from 'rxjs';
import { Product } from '../product.model';
import { ProductApiService } from '../product-api.service';

interface ProductState {
  products: Product[];
  isLoading: boolean;
  error: string | null;
}

export const ProductStore = signalStore(
  { providedIn: 'root' },

  // Initial state
  withState<ProductState>({ products: [], isLoading: false, error: null }),

  // Computed signals
  withComputed(({ products }) => ({
    productCount: computed(() => products().length),
    totalValue: computed(() => products().reduce((s, p) => s + p.price, 0)),
  })),

  // Methods - synchronous and async
  withMethods((store, api = inject(ProductApiService)) => ({
    addProduct(product: Product) {
      patchState(store, s => ({ products: [...s.products, product] }));
    },
    removeProduct(id: number) {
      patchState(store, s => ({
        products: s.products.filter(p => p.id !== id)
      }));
    },
  })),
  // rxMethod - bridge between RxJS and signal store
```

```

        loadProducts: rxMethod<void>(pipe(
            tap(() => patchState(store, { isLoading: true, error: null })),
            switchMap(() => api.getAll()),
            tap({
                next: products => patchState(store, { products, isLoading: false }),
                error: err      => patchState(store, { error: err.message, isLoading:
false })
            })
        )),
    )))
};

// product-list.component.ts - using the signal store
@Component({
    standalone: true,
    template: `
        @if (store.isLoading()) { <p>Loading...</p> }
        <p>{{ store.productCount() }} products (Total: {{ store.totalValue() |
currency }})</p>
        @for (p of store.products(); track p.id) {
            <div>{{ p.name }}<button
(click)='store.removeProduct(p.id)'+>x</button></div>
        }
    `
})
export class ProductListComponent implements OnInit {
    store = inject(ProductStore);
    ngOnInit() { this.store.loadProducts(); }
}

```

6. Choosing the Right State Management Approach

There is no single correct answer — the right tool depends on your application's complexity, team size, and requirements. Use this guide to decide:

```
// — Decision Guide —————

// Simple app / prototyping / 1-2 developers
// → Component-local state (class properties, @Input/@Output)
// → Angular Signals (signal() in components for simple reactive state)

// Medium app / shared state across 2-10 components
// → Service with BehaviorSubject (most Angular projects never need more)
// → Angular Signals in a service (inject, signal(), computed())

// Large app / complex interactions / multiple developers
// → NgRx Signal Store – less boilerplate than classic NgRx, Signal-native
// → Classic NgRx – if you need time-travel debugging, strict conventions,
//                  or your team already knows Redux

// Criteria that suggest NgRx:
// ✓ State needs to be shared across many distant (non-parent/child) components
```

```
// ✓ State transitions must be logged/auditable (finance, healthcare apps)
// ✓ You need time-travel debugging via Redux DevTools
// ✓ Large team (5+ devs) that benefits from enforced conventions
// ✓ Complex state that depends on multiple async operations

// Criteria that suggest BehaviorSubject / Signals:
// ✓ Shared state between a small number of related components
// ✓ You want zero extra dependencies
// ✓ Small-to-medium team comfortable with RxJS
// ✓ State logic is straightforward CRUD without complex interdependencies
```

For new Angular 17+ projects, Angular Signals + Signal Store is the direction the Angular team is moving toward. BehaviorSubject services remain excellent for intermediate complexity. Classic NgRx is battle-tested for enterprise apps but is increasingly being replaced by Signal Store in new projects.

7. Component Communication Patterns

State management is also about how components talk to each other. The right pattern depends on the relationship between the communicating components.

```
// — 1. Parent → Child: @Input() —————
// Direct parent passes data down
@Component({ template: `<app-child [items]='products' [title]='heading'/>` })
class ParentComponent { products = []; heading = 'My List'; }

// — 2. Child → Parent: @Output() / EventEmitter —————
// Child notifies parent of events
@Component({ template: `<button (click)='add.emit(item)'>Add</button>` })
class ChildComponent { @Output() add = new EventEmitter<Product>(); }

// — 3. Sibling communication: Shared Service —————
// Components that are not parent/child use a singleton service
@Injectable({ providedIn: 'root' })
class SharedService {
  private selected$ = new BehaviorSubject<Product | null>(null);
  readonly product$ = this.selected$.asObservable();
  select(p: Product) { this.selected$.next(p); }
}

// — 4. Signal-based communication —————
@Injectable({ providedIn: 'root' })
class SharedSignalService {
  private _selected = signal<Product | null>(null);
  readonly selected = this._selected.asReadonly();
  select(p: Product) { this._selected.set(p); }
}

// — 5. Deep tree: Router state / URL —————
// Use query params or route state to pass data between routed views
```

```
this.router.navigate(['/details'], { queryParams: { id: product.id } });
```

8. OnPush Change Detection with State

Angular's default change detection checks every component on every event. Switching to `ChangeDetectionStrategy.OnPush` tells Angular to only re-render a component when its `@Input` references change, an event originates from it, or an async pipe / signal emits a new value — dramatically improving performance in large apps.

```
import { Component, ChangeDetectionStrategy, input, signal } from '@angular/core';

// OnPush - Angular only checks this component when:
// 1. An @Input reference changes (not just the object's internal value)
// 2. An event handler inside the component fires
// 3. An Observable used via async pipe emits a new value
// 4. A signal() used in the template emits a new value
// 5. ChangeDetectorRef.markForCheck() is called manually
@Component({
  selector: 'app-product-card',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <div>
      <h3>{{ product().name }}</h3>
      <p>{{ product().price | currency }}</p>
      <p>Views: {{ viewCount() }}</p>
      <button (click)='increment()'>View</button>
    </div>
  `
})
export class ProductCardComponent {
  // Angular 17+ signal input - reactive and OnPush-friendly
  product = input.required<Product>();

  // Local signal - OnPush detects signal changes automatically
  viewCount = signal(0);

  increment() {
    this.viewCount.update(n => n + 1); // OnPush re-renders automatically
  }
}

// ⚠ Common OnPush pitfall - mutating an object does NOT trigger re-render
// WRONG: this.product.name = 'new name'; // same reference - OnPush won't
// update
// RIGHT: this.product = { ...this.product, name: 'new name' }; // new reference
```

Interview Questions & Answers

Q1: What is state management and why do Angular apps need it?

- State is any data the app needs to remember and display — user info, cart items, API results, loading flags, selected items, and UI visibility
- Without a state management strategy, the same data gets duplicated in multiple components, goes out of sync, and becomes impossible to reason about as the app grows
- State management defines a single source of truth for each piece of data, a controlled way to update it, and a reactive mechanism to notify the UI when it changes
- Angular offers a spectrum of tools from simple shared services (BehaviorSubject) to Signals (built-in) to NgRx (Redux pattern) — the right choice depends on app complexity

Q2: What is the BehaviorSubject state service pattern and how does it work?

- A singleton service holds a private BehaviorSubject and exposes derived Observables (items\$, isLoading\$, error\$) that components subscribe to via the async pipe
- State is updated only via service methods that call this.state\$.next({ ...currentState, ...changes }) — immutable spread updates prevent accidental mutation
- Components cannot call .next() directly (the subject is private) — they call semantic methods like addItem(), setLoading(), or setError()
- BehaviorSubject initialises with a value and gives new subscribers the current value immediately — making it ideal for state that components need as soon as they are created

Q3: What are Angular Signals and how do they differ from Observables?

- A signal is a reactive value container: signal(0) holds a value that notifies dependents synchronously when it changes via set() or update()
- Signals are synchronous and always hold a current value readable by calling signal() — unlike Observables which are lazy, asynchronous, and require subscription
- computed(() => ...) creates a derived signal that recalculates automatically when any signal it reads changes — similar to a memoised selector
- Signals integrate natively with Angular's change detection — no async pipe is needed in templates; no manual subscription or unsubscription is needed anywhere

Q4: What is the difference between signal.set() and signal.update()?

- set(value) — replaces the current signal value with a new value directly: count.set(10)
- update(fn) — computes the new value as a function of the current value: count.update(n => n + 1)
- Use set() when the new value is independent of the old value (e.g., resetting a form field)
- Use update() when the new value depends on the current value (e.g., incrementing a counter, toggling a boolean, immutably adding an item to an array)

Q5: What are the four main building blocks of NgRx?

- Actions — plain objects that describe an event that happened (e.g., `loadProducts`, `addToCart`, `loginSuccess`); they are dispatched by components and consumed by reducers and effects
- Reducers — pure functions that take the current state and an action and return a new immutable state; they contain zero side effects and are the only place state changes
- Selectors — memoised functions that derive slices of state from the Store; they only recalculate when their input slice changes, preventing unnecessary re-renders
- Effects — classes that listen for specific actions and perform side effects (HTTP calls, navigation, `localStorage`) before dispatching success or failure actions

Q6: What is NgRx Signal Store and when would you use it over classic NgRx?

- NgRx Signal Store (from `@ngrx/signals`) is a lightweight store built on Angular Signals — it uses `signalStore()`, `withState()`, `withMethods()`, and `withComputed()` instead of Actions/Reducers/Effects
- It dramatically reduces boilerplate — no action creators, no reducers file, no selectors file; state is updated directly via `patchState()`
- Use Signal Store for new Angular 17+ projects that want NgRx conventions (immutable state, encapsulated logic) without the ceremony
- Use classic NgRx when you need strict audit trails, the Redux DevTools time-travel debugging workflow, or your team is already deeply familiar with the classic pattern

Q7: What is `ChangeDetectionStrategy.OnPush` and how does it relate to state management?

- `OnPush` tells Angular to skip re-rendering a component unless: an `@Input` reference changes, an event fires from within it, an async pipe emits, or a signal in the template changes
- It is a major performance optimization — without it, Angular checks every component on every user event, even if the component's data did not change
- `OnPush` works naturally with `BehaviorSubject` + async pipe (a new emission triggers detection) and with Signals (Angular tracks signal reads in templates automatically)
- The critical pitfall: mutating an object directly (`this.items.push(...)`) does NOT trigger `OnPush` re-render because the array reference is the same — always return a new reference

Q8: What is the difference between `toSignal()` and `toObservable()`?

- `toSignal(observable$)` converts an Observable into a Signal — Angular auto-subscribes and auto-unsubscribes; the signal always holds the latest emitted value
- `toObservable(signal)` converts a Signal into an Observable — useful when you need to pipe a signal through RxJS operators like `debounceTime` or `switchMap`
- Both are part of `@angular/core/rxjs-interop` and must be called within an injection context (constructor or field initializer)
- `toSignal()` requires an `initialValue` option if the Observable does not emit synchronously — this is the value the signal holds before the first emission arrives

Q9: What are the different component communication patterns in Angular?

- Parent to Child: `@Input()` — parent passes data down via property binding; use the new `input()` signal function in Angular 17+ for signal-based inputs
- Child to Parent: `@Output()` with `EventEmitter` — child emits events that the parent listens to; for signal-based outputs, use `output()` in Angular 17+
- Sibling / Distant components: shared singleton service — inject the same service into both components; use `BehaviorSubject` or signal to hold shared state
- Router-connected views: Router state, query params, or route data — pass data through the URL for bookmarkable state that survives page refresh

Q10: How do you decide which state management approach to use for a new Angular project?

- Start simple: use component-local signals or class properties for UI state that is not shared. Use `@Input/@Output` for direct parent-child communication
- Scale up to a shared service with `BehaviorSubject` or signals when two or more unrelated components need the same data — this covers the majority of real-world apps
- Reach for `NgRx Signal Store` when state logic is complex enough to benefit from encapsulation, there are multiple developers, or you want enforced conventions and devtools support
- Reserve classic `NgRx` for enterprise-scale apps requiring strict auditability, time-travel debugging, a rich ecosystem of middleware, or teams already fluent in the `Redux` pattern